

Assumption University
Department of Computer Science (VMS)

Senior Project 2
CSX3011

6238135 - Zwe Min Maw
6238001 - Ahmad Yasi Faizi
6318013 - Saw Zwe Wai Yan

PS Park - Apartment Management System and Dashboard Analysis

Department of Computer Science, Vincent Mary School of Science and Technology

Project Advisor

Asst. Prof. Dr. Rachuda Setthawong

Project Committee

Mr. Chayapol Moemeng
Asst. Prof. Dr. Anilkumar Kothalil Gopalakrishnan

Project Title:	PS Park - Apartment Management System and Dashboard Analysis
By:	6238135 Zwe Min Maw
	6238001 Ahmad Yasi Faizi
	6318013 - Saw Zwe Wai Yan
Project Advisor:	Asst. Prof. Dr. Rachsuda Setthawong
Academic Year:	2/2023

This was authorized by the senior committees of the computer science department at the Vincent Mary School of Science and Technology at Assumption University. Senior Project Two is summited in partial fulfillment for the degree of Bachelor of Science in Computer Science.

Approval Committees:

(Asst. Prof. Dr. Rachsuda Setthawong)

Project Advisor

---	---
(Mr. Chayapol Moemeng)	(Asst. Prof. Dr. Anilkumar Kothalil Gopalakrishnan)
Committee Member	Committee Member

Acknowledgement

We would like to express our heartfelt gratitude to Asst. Prof. Dr. Rachsuda Setthawong, Asst. Prof. Dr. Pisal Setthawong, and the Apartment Manager, Mr. Pishan Setthawong, for their guidance and support throughout our senior project. Their expert advice, encouragement, and timely feedback have been instrumental in making our project a success. It helped us identify key areas of importance and guided us on what to pursue and what to avoid. The weekly meetings provided us with a clear roadmap for our next steps, and their review of our final page functionality was precise and detailed.

We would also like to extend our thanks to Mr. Chayapol Moemeng and Asst. Prof. Dr. Anilkumar Kothalil Gopalakrishnan, who willingly accepted to be our committee members. Their valuable insights and constructive feedback have greatly improved the quality of our project.

We are also grateful to our friends who were part of this project, supporting us and providing valuable feedback. Their contributions were crucial in shaping our project and ensuring its success.

Finally, we express our sincere appreciation to Assumption University for providing us with the necessary resources and support to complete this project. This opportunity has been invaluable, and the knowledge gained from this experience will undoubtedly aid us in our ongoing learning and growth.

Abstract

PS Park - Apartment Management System and Dashboard Analysis is a comprehensive software solution designed to enhance the operational efficiency of apartment complexes and residential communities. This application integrates advanced features to address the diverse needs of property managers, residents, and maintenance personnel, fostering a cohesive and well-managed living environment. The application encompasses a range of functionalities, including resident management, facility booking, maintenance requests, apartment feedback, financial analysis, and much more. Resident management features facilitate seamless tenant onboarding, lease management, and payment processing, ensuring a smooth administrative workflow for property managers. The application also incorporates a robust maintenance module, allowing manager to list down report issues, while efficiently allocate resources and track the resolution process. Moreover, the PS Park application contains a dashboard that can be accessed for financial analysis and resource management. Security is a top priority, with the implementation of secure access controls, such as login and password. PS Park - Apartment Management System and Dashboard Analysis Application stands as an indispensable tool for modern property management, offering a user-friendly and secure platform that seamlessly integrates various functions to optimize residential community operations.

Table of Contents

Chapter 1: Introduction 8

Chapter 2: Background 9

Chapter 3: User Interface/Experience (UI/UX)..... 18
UI/UX Design..... 18
Design Feedback 19

Chapter 4: System Requirements 20
Functional Requirement 21

Chapter 5: Analysis 22
Cost and Benefit Analysis 23

Chapter 6: System Design 24
System Analysis and Design 25

Chapter 7: Database Design 26
Database Design of Proposed Project 27

Chapter 8: Interface Design 34
Final Version Interface Design 35

Chapter 9: Conclusion and References 38
Conclusion 39
References 40

List of Figures

Figure 3.1 Navigation Bar
Figure 3.2 Analysis Dashboard
Figure 3.3 Management Dashboard
Figure 3.4 Check In
Figure 3.5 Check Out
Figure 3.6 Rate Maintenance
Figure 3.7 Rate Maintenance Details
Figure 3.8 Room Maintenance
Figure 3.9 Room Maintenance Details
Figure 3.10 Tenant Maintenance
Figure 3.11 Tenant Maintenance Details
Figure 3.12 Summary Billing Report
Figure 3.13 Summary Meter
Figure 3.14 Generate Bill
Figure 3.15 Contract Generation
Figure 3.16 Printing Payment
Figure 3.17 Request
Figure 3.18 Profile Account
Figure 6.1 Ghanttchart of beginning of the project until its end
Figure 7.1: Tenants Table
Figure 7.2: Tenants Table's Relationships
Figure 7.3: Contacts Table
Figure 7.4: Contacts Table's Relationship
Figure 7.5: Addresses Table
Figure 7.6: Addresses Table's Relationship
Figure 7.7: RoomBaseDetails Table
Figure 7.8: RoomBaseDetails Table's Relationships
Figure 7.9: RoomStatusDetails Table
Figure 7.10: RoomStatusDetails Table's Relationship
Figure 7.11: Tenancy_records Table
Figure 7.12: Tenancy_records Table's Relationships
Figure 7.13: Rates Table
Figure 7.14: Rates Table's Relationship
Figure 7.15: Room_rates Table
Figure 7.16: Room_rates Table's Relationship
Figure 7.17: Meter_readings Table
Figure 7.18: Meter_readings Table's Relationship
Figure 7.19: Requests Table

Figure 7.20: Requests Table’s Relationship
Figure 7.21: Bills Table
Figure 7.22: Bills Table’s Relationships
Figure 7.23: GeneratedbillRecords Table
Figure 7.24: GeneratedbillRecords Table’s Relationships
Figure 7.26: Manager Table
Figure 7.27: Manager Table’s Relationship
Figure 7.28: Service_Contacts Table
Figure 7.29: Service_Contacts Table’s Relationship
Figure 8.1 Sign-In Interface
Figure 8.2 Dashboard (Management)
Figure 8.3 Dashboard (Analysis)
Figure 8.4 Rate Table Interface
Figure 8.5 Add New Rate/Item Interface
Figure 8.6 Room Table Interface
Figure 8.7 Add New Room Interface
Figure 8.8 Room Item Selection Interface
Figure 8.9 Tenant Table Interface
Figure 8.10 Tenant Maintenance Interface
Figure 8.11 Tenant Maintenance Interface Edit
Figure 8.12 Check-In
Figure 8.13 Check-Out
Figure 8.14 Contract Generation
Figure 8.15 Contract Generation (Contract Options)
Figure 8.16 The PDF Generator
Figure 8.17 Generate Billing
Figure 8.18 Billing Details
Figure 8.19 Payment Generation
Figure 8.20 Billing Report
Figure 8.21 Meter and Water
Figure 8.22 Request Maintenance
Figure 8.23 [Interface for Adding New Maintenance Request]
Figure 8.24 Profile

Chapter One

1. **Introduction:** The PS Park - Apartment Management System and Dashboard Analysis is a meticulously crafted web application tailored to meet the unique requirements of PS Park apartment, offering an innovative solution for efficient property management. as the It is a sophisticated software encompasses a wide array of advanced features outlined in the abstract. Its main objective is to notably improve the operational efficiency of apartment complexes and residential communities, promoting well-organized living space for property managers and residents alike.
2. **Statement of the Problem:** The current property management system at PS Park apartment is not just dealing with operational issues, but it's also giving us a hard time with its user interface (UI). The current UI design is not very user-friendly, making it tough for the user to smoothly find and use essential features. This makes our overall experience with the system not as great as it could be.

   And it's not just us the developers – property managers are also having a tough time navigating through complicated processes because of this not-so-friendly interface. We need a UI that's easier to use, looks good, and makes it simple for us to ask for maintenance help or engage with the system. The current situation shows that not only do we have to add new features such as analysis and management but to also fix the not-so-great UI.

   So, a solution should not only focus on fixing the operational stuff but also on making the interface more user-friendly. We all need a system that's not just efficient but also feels good to use – for both the developers and the property managers.

Chapter Two

Objective: the primary goal of this project is to conceive and create a user-friendly apartment management application named "PS Park," specifically tailored for PS Park apartment manager to manage and maintain the apartment. The following objectives have been identified for this application.

1. **Streamline Property Management Processes:** In order to enhance the operational efficiency of PS Park's property management, a critical focus will be placed on streamlining existing processes. This involves a comprehensive analysis of the current workflows, identifying redundancies, and implementing automation where applicable. By leveraging advanced technologies and integrating features outlined in the abstract, the goal is to create a seamless and efficient property management system. This includes optimizing resident onboarding, lease management, and payment processing, ensuring that property managers can navigate administrative tasks with greater ease and speed.
2. **Ensure Security Measures:** Security is paramount in any property management system, and addressing this concern will involve a meticulous evaluation and enhancement of the current security measures. This includes a detailed assessment of login and password features. By prioritizing security, we aim to safeguard sensitive resident and property information, instilling confidence among users in the reliability and trustworthiness of the system. A secure foundation is essential for the successful functioning of any property management application.
3. **Encourage Financial Transparency:** Financial transparency is vital for effective decision-making and resource management within PS Park. The objective here is to integrate financial analysis tools into the property management system, providing property managers with clear insights into budgeting, expenses, and revenue. This transparency not only enhances the overall financial health of the community but also fosters trust among residents. Through easy-to-access financial information, property managers can make informed decisions, promoting a well-balanced allocation of resources and contributing to the overall success of PS Park's property management.

External Exposure: The exposure was obtained with the assistance of Ajarn Rashuda, allowing us to connect with a manager of an apartment and gather details and information from him concerning his existing apartment application. Through this connection, we successfully extracted essential changes that the manager desired and received valuable input for new features.

Chapter Three

Figma UI design: In this section of our report, we are excited to provide a preview of the user interface (UI) we have meticulously crafted using Figma. Within these visuals, we have seamlessly integrated various elements that define the user experience, ranging from the login and signup screens to the essential financial analyse and management of the dashboard.

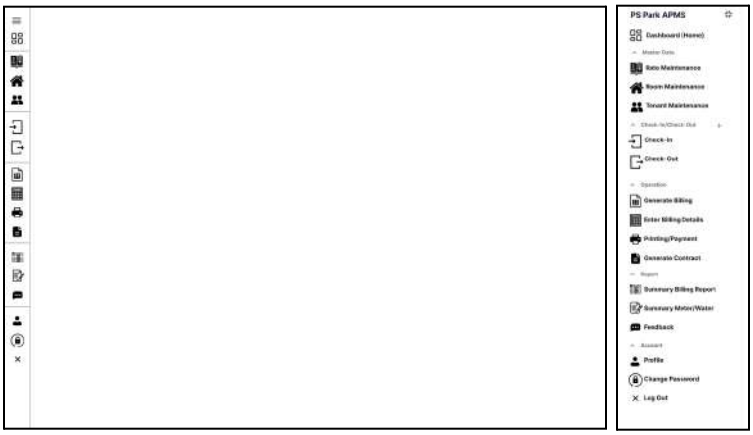


Figure 3.1 Navigation Bar, from a menu list to a dashboard we allow the user to have a much efficient and convenient access to all essential menus to navigate throught the application.

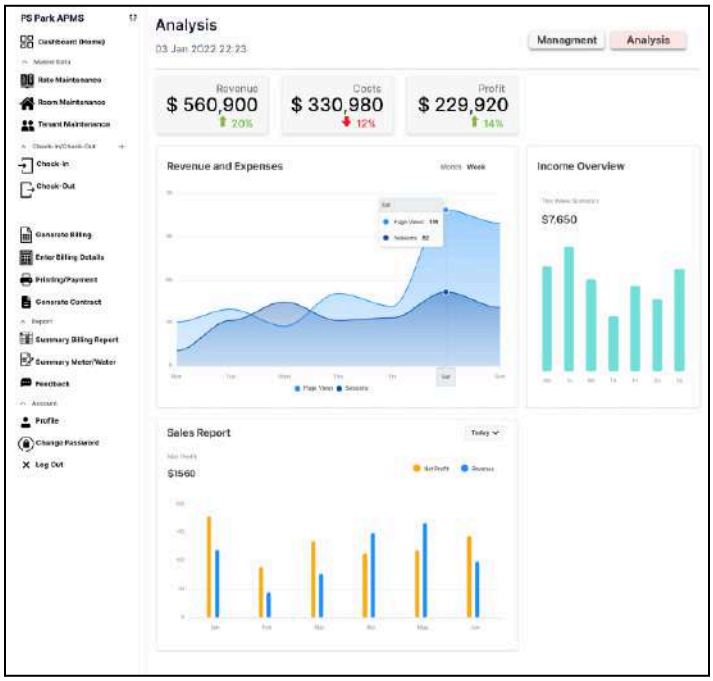


Figure 3.2 Analysis Dashboard, consist of all financial analysis

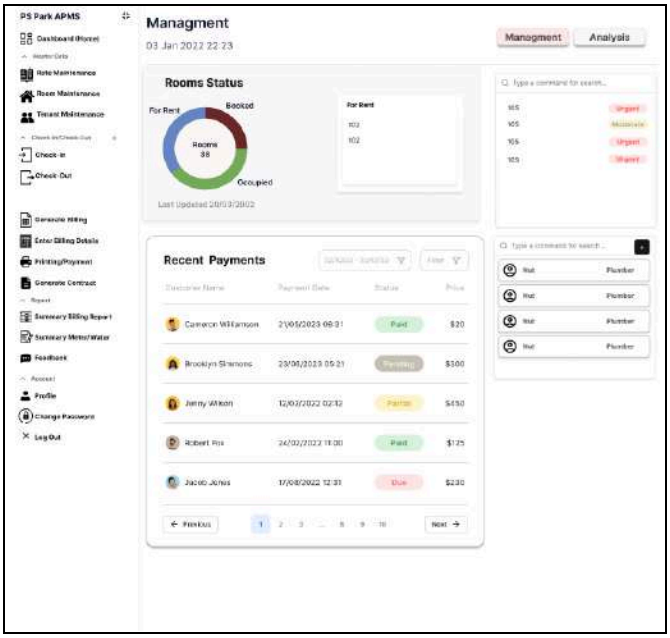


Figure 3.3 Managment Dashboard, consist of all financial and resource management for the manager

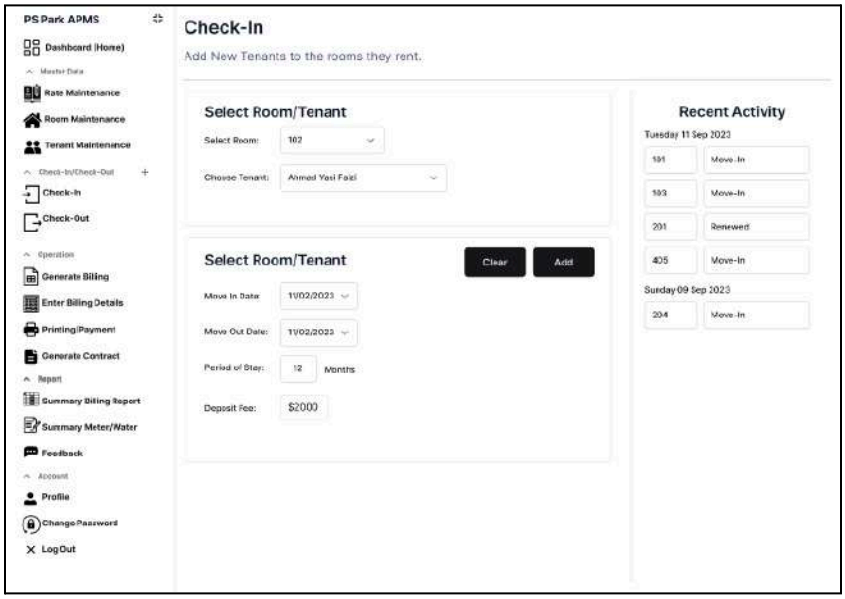


Figure 3.4 Check In, consist of resident check in and all the items and services that the tenant would want it in his room.

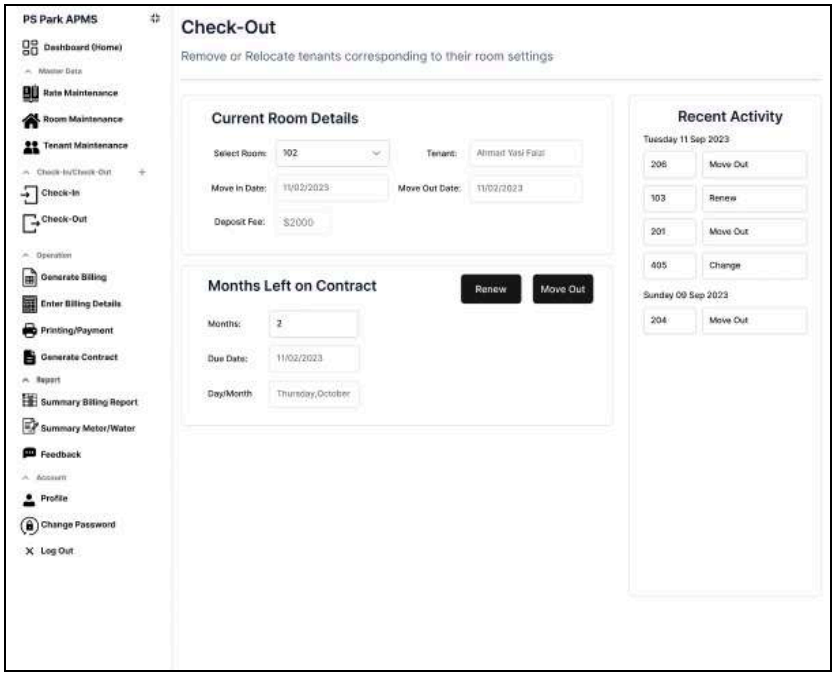


Figure 3.5 Check Out, consist of resident check in and all the items and services that the tenant would want it in his room.

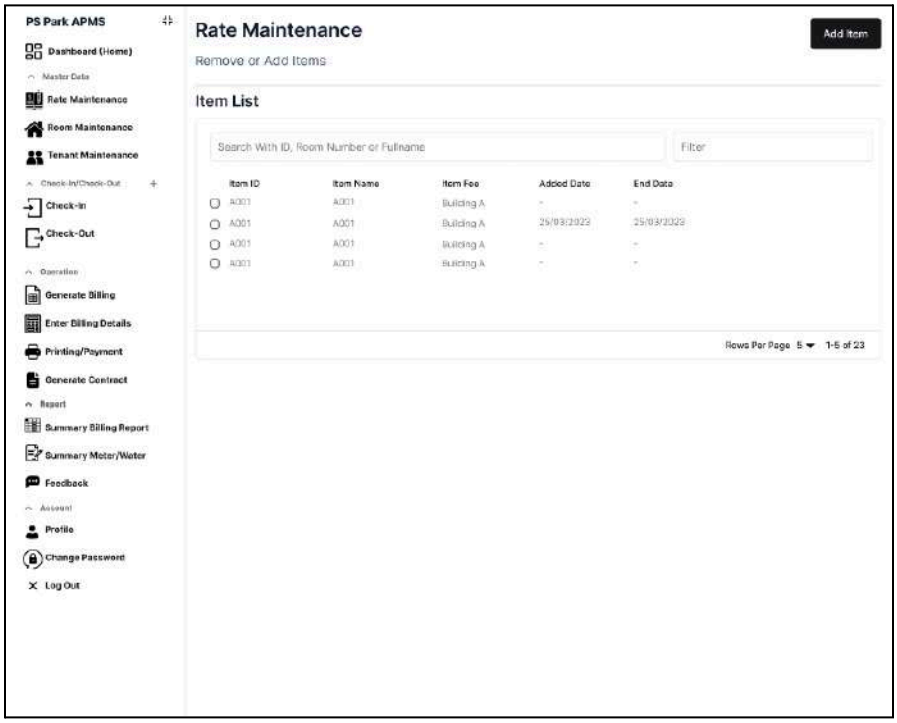


Figure 3.6 Rate Maintenance, consist of room and tenant item maintenance we can add or remove items from room

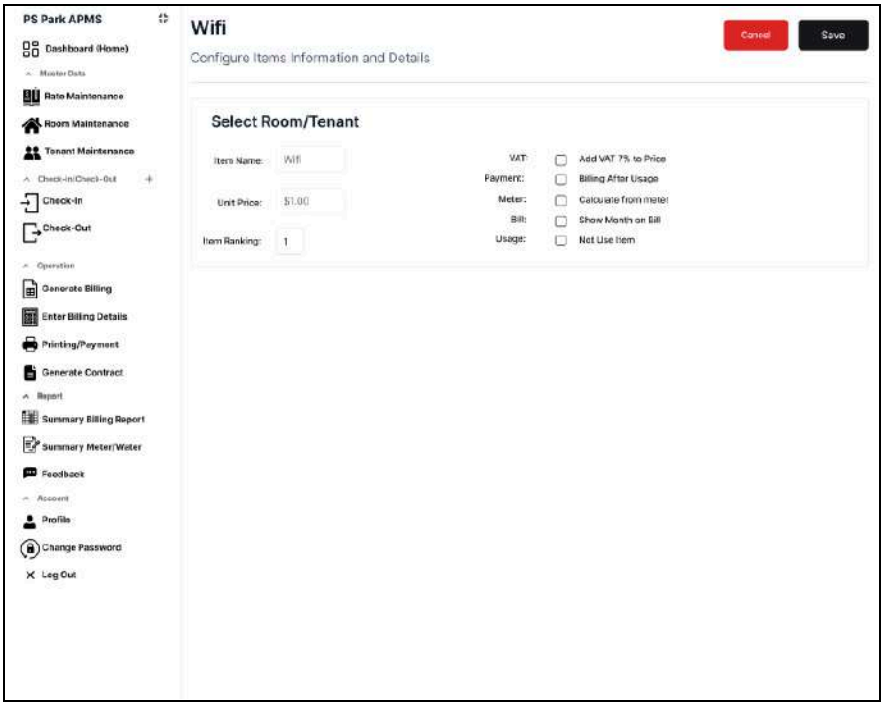


Figure 3.7 Rate Maintenance details, consist of room and tenant item maintenance we can add or remove items from room

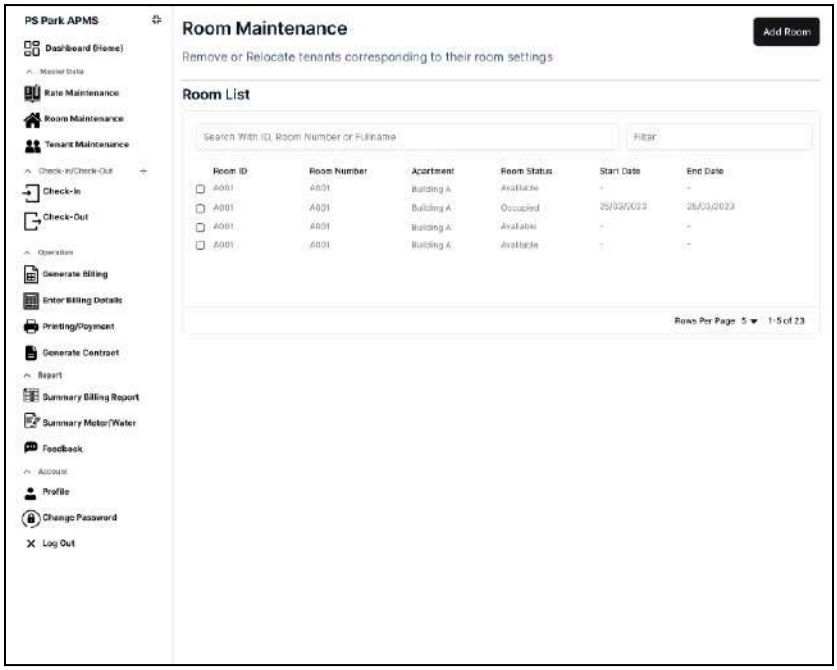


Figure 3.8 Room Maintenance, consist of room management check rents, faults, etc.

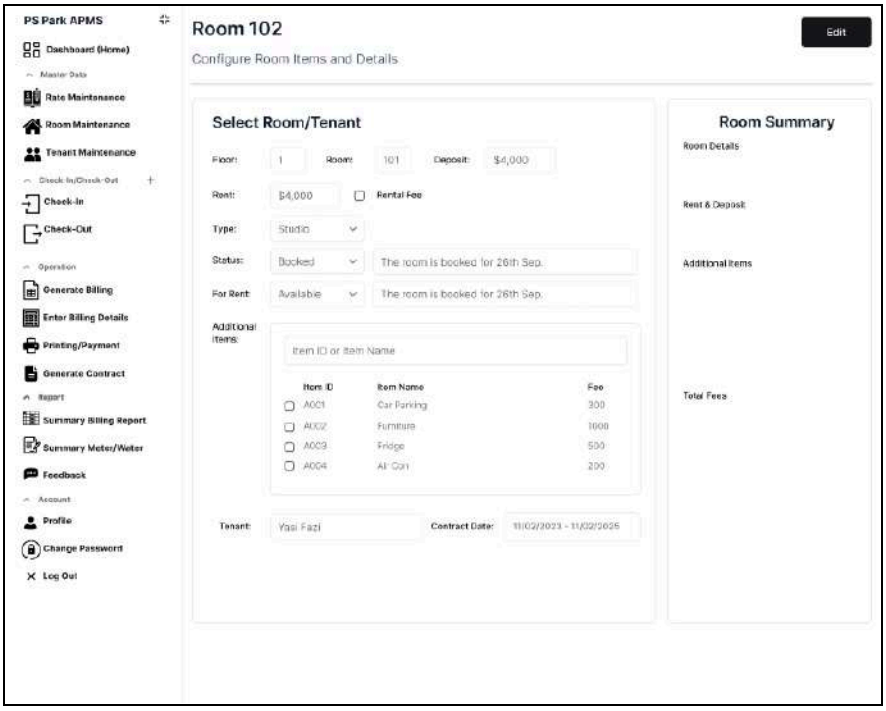


Figure 3.9 Room Maintenance Details, consist of room management check rents, faults, etc.

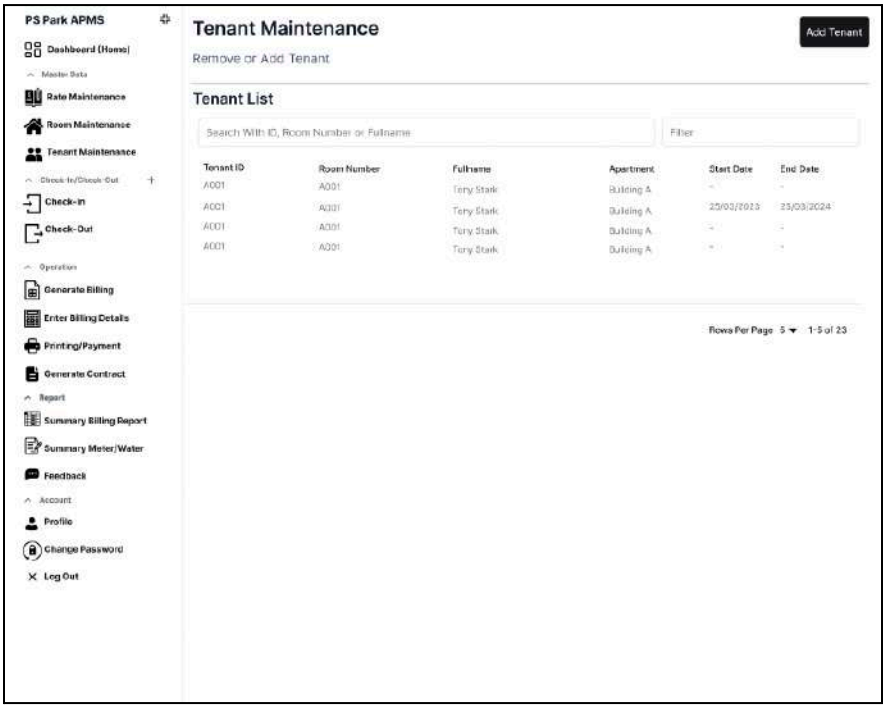


Figure 3.10 Tenant Maintenance, consist of Tenant management check resident info, rents, item usage and more.

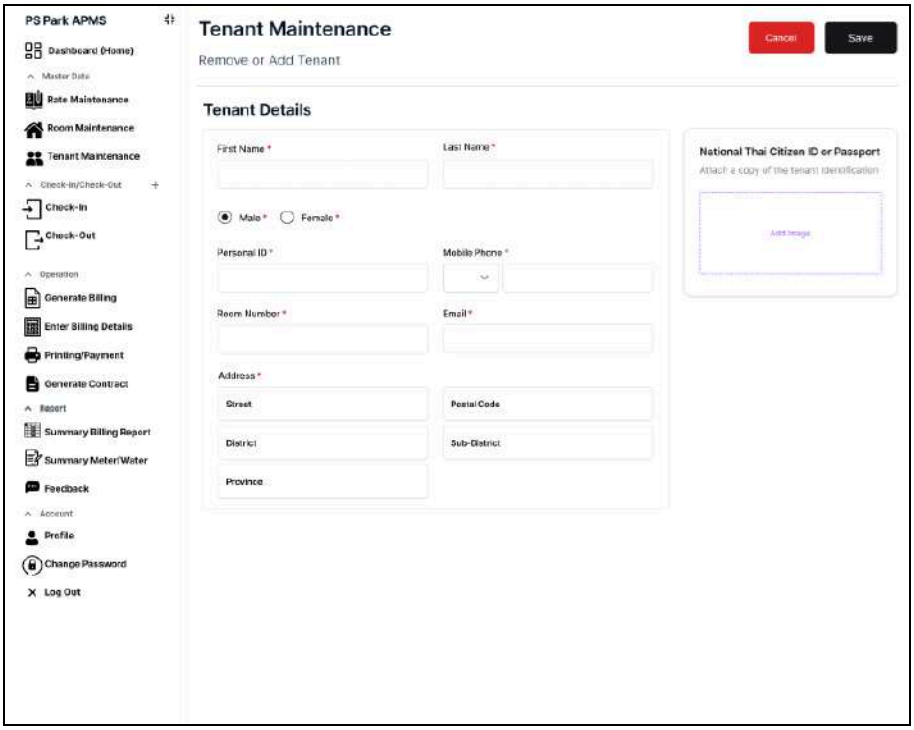


Figure 3.11 Tenant Maintenance Details, consist of Tenant management check resident info, rents, item usage and more.

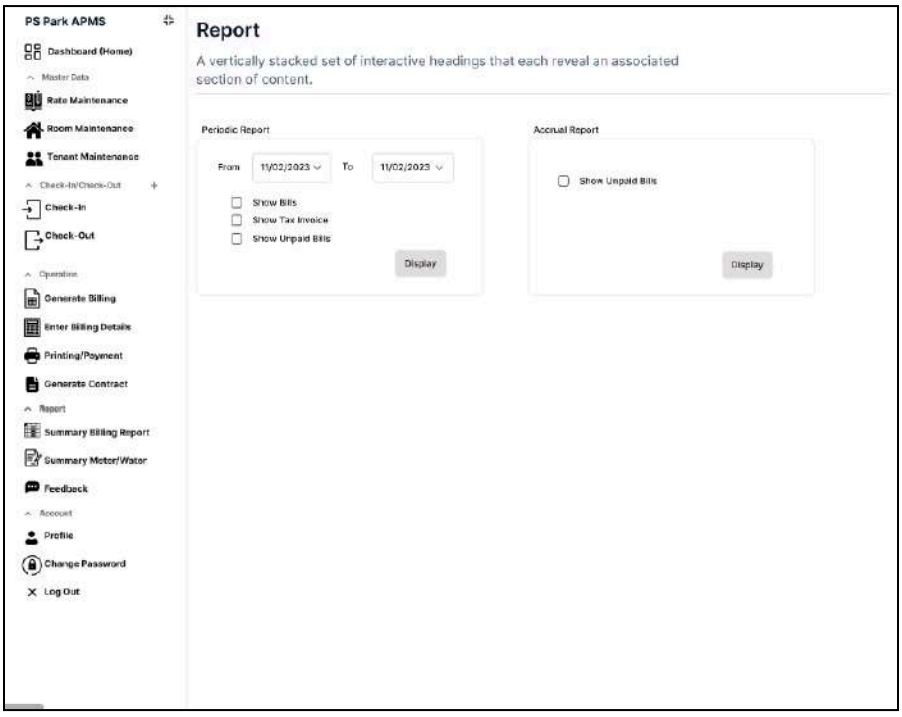


Figure 3.12 Summary Billing Report, reporting bill and it full summary would be uploaded from here.

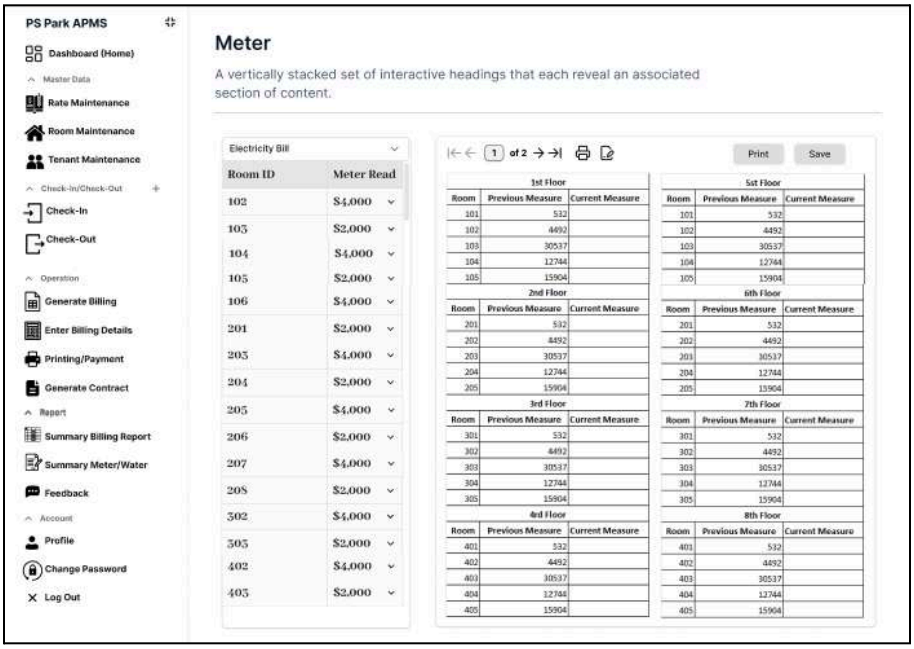


Figure 3.13 Summary Meter, we can view the electricity meter of each rooms in an apartment.

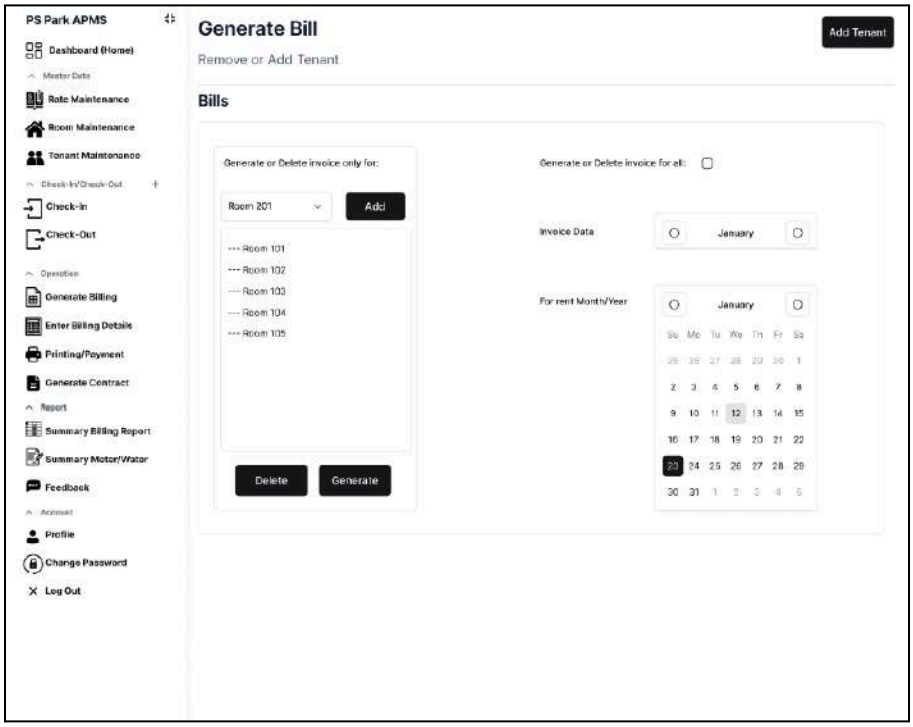


Figure 3.14 Generate Bill, we can generate and create bill for each rooms.

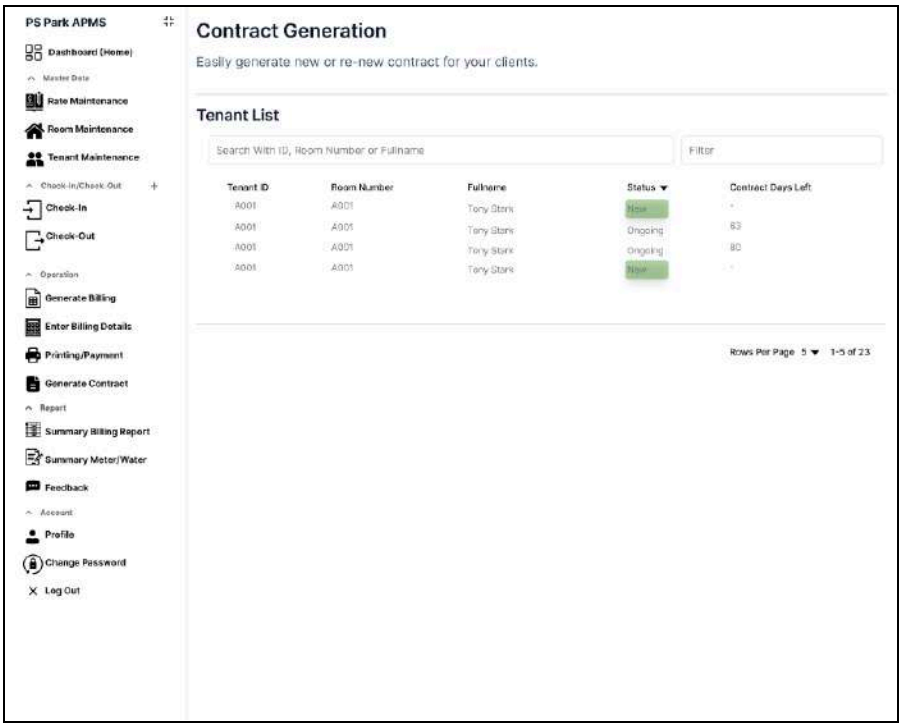


Figure 3.15 Contract Generation, create or generate contract for a room.

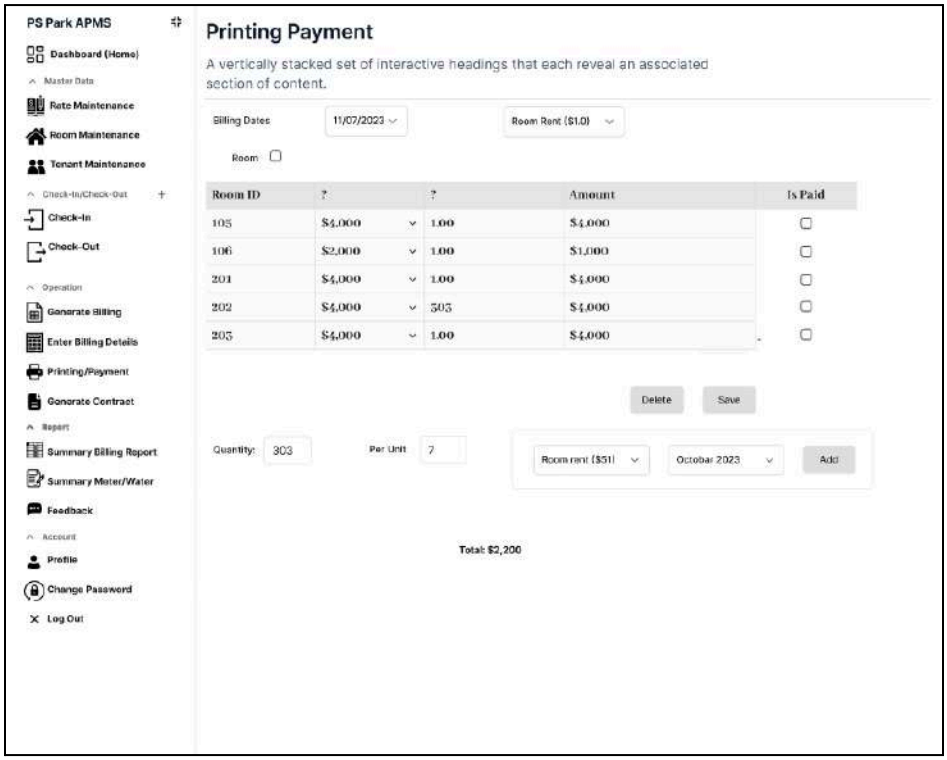


Figure 3.16 Printing Payment, we can print the bill for the tenant.

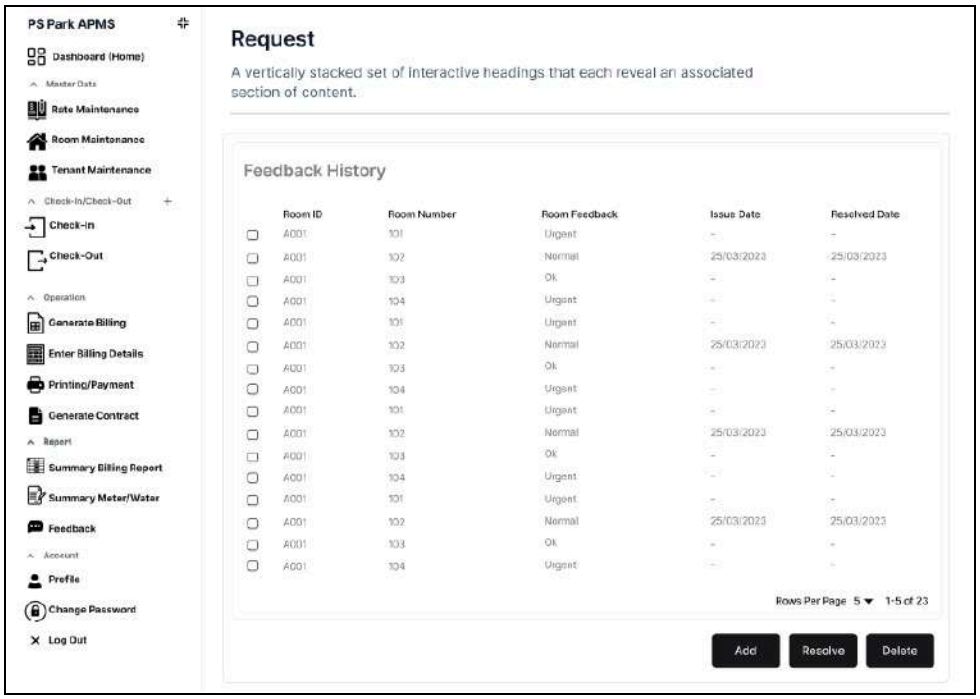


Figure 3.17 Request, it's the response for any item within the room, bed, toilet, shower, walls, etc. And if there is any issue for it to be fixed..

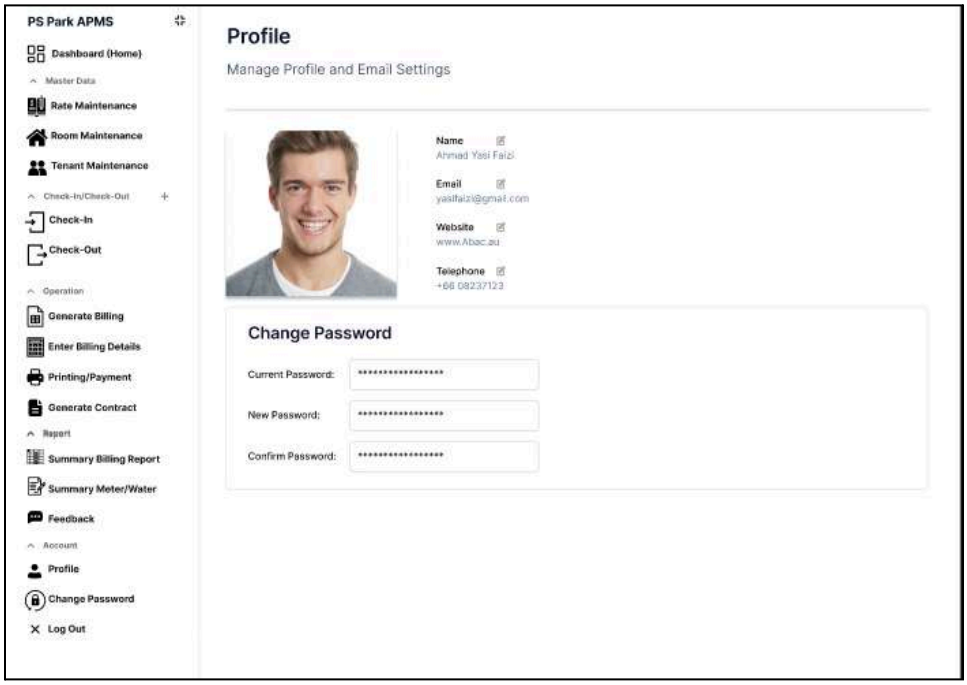


Figure 3.18 Profile account, This is manager account where he can add new password and take care of its security measures..

Design Feedback: Following recent updates to the PS Park - Apartment Management System's user interface, feedback from the management indicated significant satisfaction with the enhancements. The focus of these updates was on improving both usability and visual appeal, making the system more user-friendly and engaging. Management specifically noted the positive impact of the changes on certain pages, where streamlined navigation and enhanced functionality made operations more efficient and less time-consuming. Additionally, they offered constructive suggestions for further minor adjustments to optimize user interaction and the presentation of information.

Chapter Four: Functional Requirements

- **Manager**

In the PS Park apartment system management, the role of the user is exclusively designated for managers who wield comprehensive control over every aspect of the system. As the sole user, the manager is vested with authority to oversee and execute various functions critical to efficient apartment management. This includes but is not limited to monitoring tenant requests, handling financial transactions, managing property maintenance, and ensuring seamless communication between the management and tenants. The functional requirements outlined in this chapter are tailored to empower managers with the tools and capabilities needed to navigate and govern the apartment system effectively. Through a user-centric approach, the system aims to streamline managerial responsibilities, enhance decision-making processes, and foster an environment conducive to optimal apartment management.

Chapter Five: Cost and Benefit Analysis

Backend Software: For backend Pyramid was chosen for its simplicity and Python-based environment, and with more feature-rich Elixir. It’s comprehensive suite of tools accelerates development with its built-in features, while concurrency and fault tolerance are invaluable for a high-performance, real-time application. This blend of technologies ensures scalability and efficient development workflows.



Frontend Software: In the realm of frontend development, React's component-based architecture had been our primary choice for its modularity and vibrant ecosystem. To further enhance our application's performance and user experience, we have opted to integrate Next.js, which will facilitate server-side rendering and static site generation. This approach, along with the HTML5, MUI and Nextron for responsive design, will form the cornerstone of our user interface, ensuring a seamless and interactive user experience.



Database: After considering various database solutions, PostgreSQL stands out as the most suitable choice for our needs. Its robustness, support for complex data structures, and ACID compliance make it an optimal solution for the dynamic and data-intensive nature of our application. PostgreSQL's ability to handle high volumes of transactions and its extensibility make it a future-proof choice for our database needs.



Development Tools: To support our development process, we will employ Jira and GitHub as our primary development tools. Jira will provide us with agile project management capabilities, enabling us to track progress, prioritize tasks, and distribute work effectively among team members. GitHub will serve as our code repository and collaboration platform.



Benefit Analysis: The "PS Park Apartment Application System" harnesses Elixir in the backend to deliver a robust, real-time service for residents, handling everything from rent payments to maintenance requests. With a Next.js-enhanced frontend, users enjoy a fast and intuitive experience, while PostgreSQL ensures secure and reliable data management. The use of Jira and GitHub in development underpins a responsive, user-focused platform that scales seamlessly with the community, streamlining property management and enhancing the residential experience.

Chapter Six: System Analysis and Design

- **Project Identification and Selection:** The core objective is to address the unique challenges faced by
- **Project Identification and Selection:** This phase involves a meticulous examination of the challenges faced by PS Park and a thorough understanding of how our proposed system can address these issues. We're not just solving problems; we're aiming to enhance the overall system design and how it operates for property managers. Project identification means ensuring that our goals align seamlessly with the needs of PS Park, making this initiative a purposeful and beneficial endeavor.
- **Project Initiation and Planning:** project initiation involves defining the scope, setting objectives, gathering information from the managers and assembling the team that will bring our vision to life. We're not just jumping in blindly; we're crafting a roadmap that guides us through the complexities of development.



Figure 6.1 Ghanttchart of beginning of the project until its end.

Based on Figure 6.1, we can observe that we initially start by meeting with the client and securing our external exposure. Following this, we proceed with working on the wireframe, UI/UX, implementing and testing the standalone web, and creating the database design, all within the month of May. Front-end development and database management then follow, concluding in the beginning of January. The backend development, along with report creation and testing, comes last and extends until the end of February, concluding by the end of March.

Chapter Seven: Database Design of Proposed Project

System Analysis and Design

In this PostgreSQL database for the PS Park apartment management system, we have structured the data into several interconnected tables, each with its own set of fields. The tables and their relationships are designed to represent and manage the various aspects of property management effectively.

Tenants Table

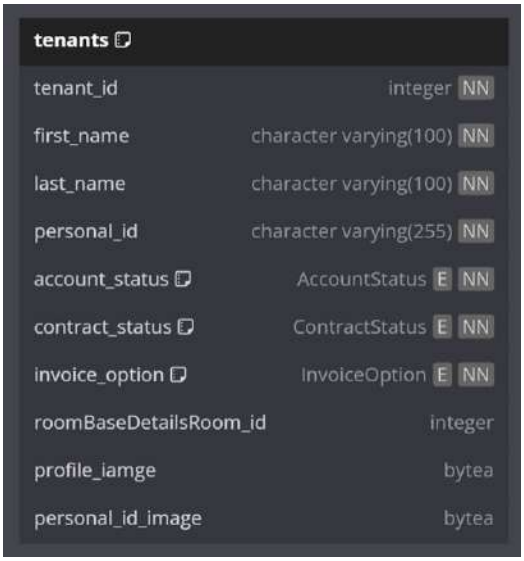


Figure: 7.1 Tenants Table

Tenants Table Fields:

tenant_id: Primary key, integer, unique identifier for each tenant.
first_name: Tenant's first name, varchar(100).
last_name: Tenant's last name, varchar(100).
personal_id: Personal ID for tenant, varchar(255).
account_status: Indicates if account is 'ACTIVE' or 'INACTIVE', enum type.
contract_status: Status of tenant's contract, enum type with values like 'NEW', 'ONGOING', 'Due', 'Warning', 'Checked Out' and 'Terminated'.
invoice_option: Tenant's preferred invoice method, enum type with 'EMAIL', 'PAPER', and 'BOTH'.
roomBaseDetailsRoom_id: Foreign key to RoomBaseDetails table, integer. profile_iamge:

Binary data for tenant's profile image, bytea type.
personal_id_image: Binary data for tenant's ID document image, bytea type



Figure: 7.2 Tenants Table's Relationships

Tenants - Contacts Relationship:

There is a one-to-one relationship between tenants and contracts. The tenant_id in contacts is a foreign key that references tenant_id in tenants. This means each tenant can have one contact record associated with them.

Tenants – Addresses Relationship:

This table is also related to the tenant's table through the tenant_id. Each tenant has one address associated with them, which suggests a one-to-one relationship.

Tenants - Tenancy_records Relationship:

There is a one-to-many relationship between tenants and tenancy_records. The tenant_id in tenancy_records references tenant_id in tenants. Each tenant may have multiple tenancy records, which could represent different periods of stay or different rooms rented over time.

Tenants - RoomBaseDetails Relationship:

This table is related to the tenants table via the roomBaseDetailsRoom_id field in the tenants table, which is a foreign key to the room_id in RoomBaseDetails. This suggests that each tenant is associated with one room.

Contacts Table



Figure 7.3 Contacts Table

Contacts Table fields:

contact_id: Unique identifier for each tenant's contact.
tenant_id: Identifier for the associated tenant.
phone_number: Tenant's phone number.
email: Tenant's email address.
line_id: Tenant's Line ID
eme_name: Emergency contact's name for tenant.
eme_phone: Emergency contact's phone number for tenant.
eme_line_id: Emergency contact's Line ID for tenant.
eme_relation: Tenant's relationship with the emergency contact.

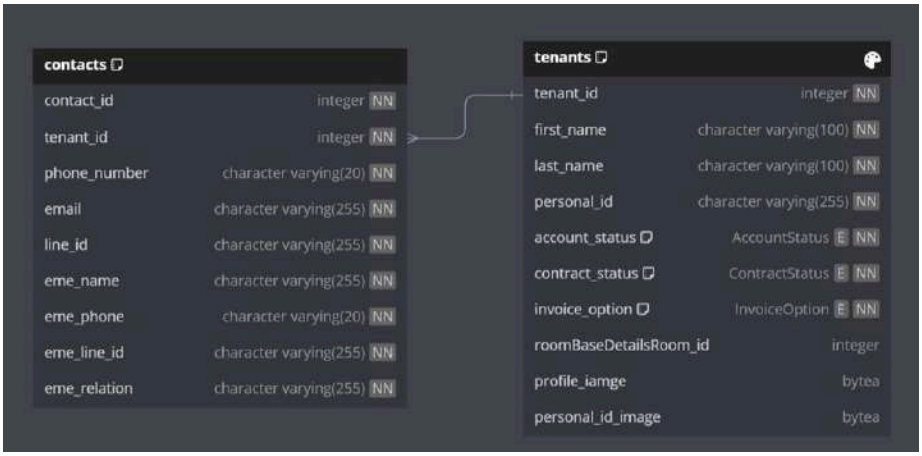


Figure: 7.4 Contacts Table's Relationship

Contacts – Tenants Relationship:

There is a one-to-one relationship between tenants and contracts. The tenant_id in contacts is a foreign key that references tenant_id in tenants. This means each tenant can have one contact record associated with them.

Addresses Table



Figure: 7.5 Addresses Table

Addresses Table Fields:

address_id: Unique identifier for each address.
tenant_id: Identifier for the associated tenant.
street: Street address.
sub_district: Sub-district or neighborhood.
district: District.
province: Province or state.
postal_code: Postal code.

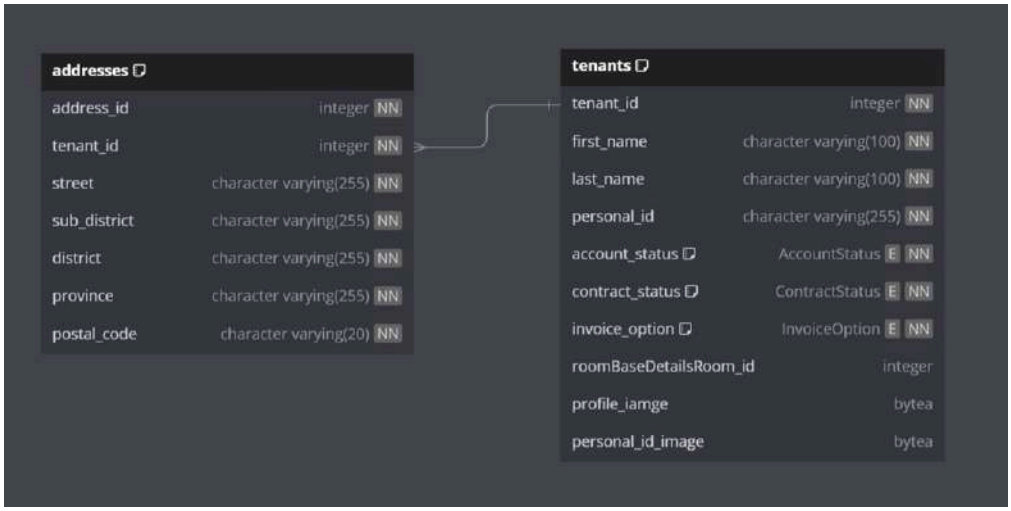


Figure 7.6 Addresses Table's Relationship

Addresses – Tenants Relationship:

This table is also related to the tenant's table through the tenant_id. Each tenant has one address associated with them, which suggests a one-to-one relationship.

RoomBaseDetails Table

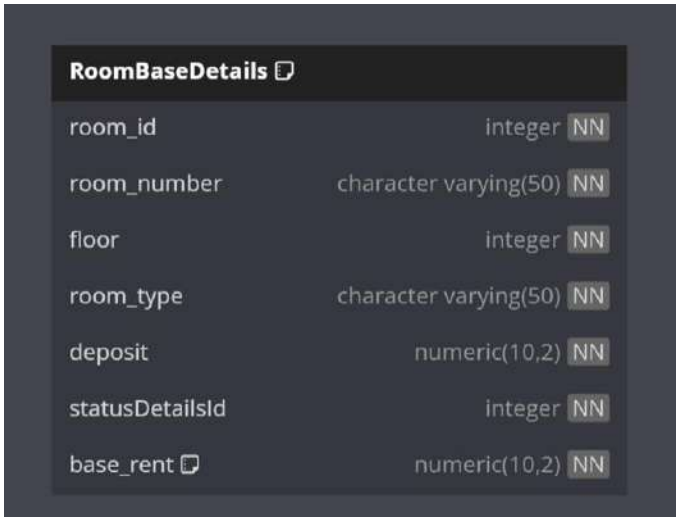


Figure 7.7 RoomBaseDetails Table

RoomBaseDetails Table Fields:
room_id: Primary key, integer, unique identifier for each room.
room_number: Identifier for the room, varchar(50).
floor: Which floor the room is on, integer.
room_type: Type or category of the room, varchar(50).
deposit: Security deposit amount for the room, numeric(10,2).
statusDetailsId: Foreign key that references RoomStatusDetails, integer.
base_rent: Monthly rent for the room, numeric(10,2).

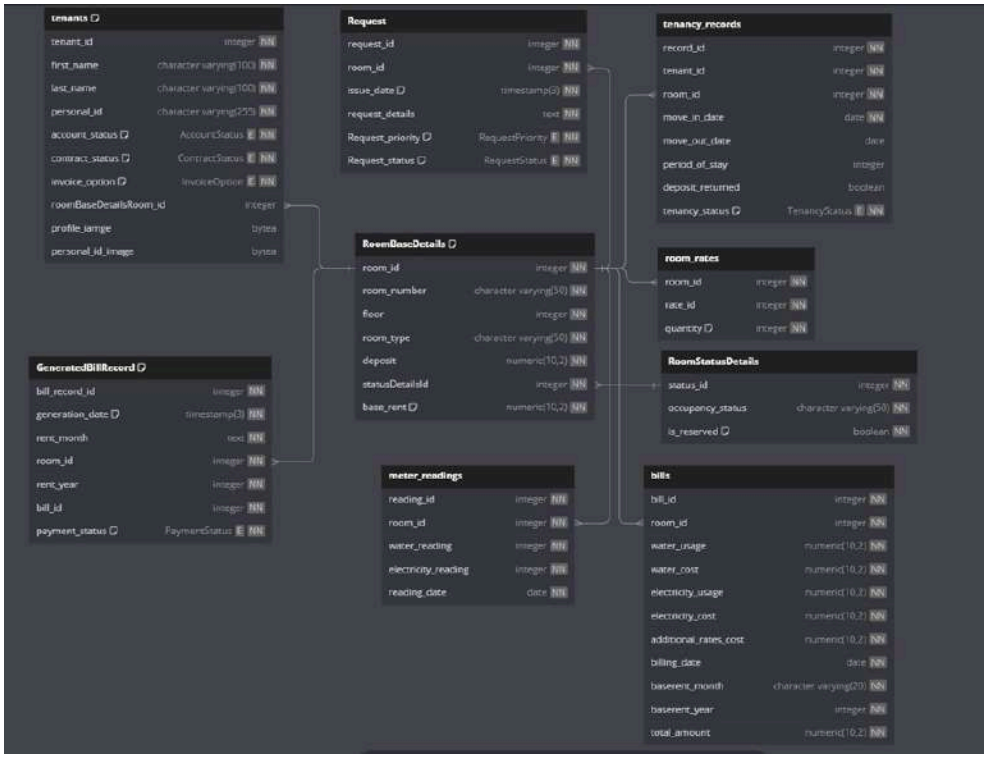


Figure 7.8 RoomBaseDetails Table's Relationships

RoomBaseDetails – Tenancy_records Relationship:

The tenancy_records table contains a room_id foreign key, creating a one-to-many relationship with the RoomBaseDetails table. This allows the database to record the history of all tenancies for each room, such as different tenants' move-in and move-out dates, which is crucial for historical data analysis and room availability management.

RoomBaseDetails – Tenants Relationship:

The tenants table has a foreign key roomBaseDetailsRoom_id that links to the room_id in the RoomBaseDetails table. This one-to-one relationship suggests that for each tenant, there is a unique room associated, allowing the system to track which room is occupied by which tenant.

RoomBaseDetails - Room_rates Relationship:

There is a one-to-many relationship established by the room_id foreign key in the room_rates table. This indicates that each room can have various rates applied over time, possibly to account for peak pricing, discounts, or other rate adjustments.

RoomBaseDetails – RoomStatusDetails Relationship:

Through the statusDetailsId field in the RoomBaseDetails, which corresponds to the status_id in the RoomStatusDetails table, a one-to-one relationship is created. Each room is linked to a single status detail record, which would include information on whether the room is occupied, vacant, or reserved.

RoomBaseDetails – GeneratedbillRecords Relationship:

The GeneratedBillRecord table has a room_id foreign key linking to the RoomBaseDetails table in a one-to-many fashion. This enables the tracking of all billing records generated for each room, such as rent, utilities, and other charges, which will be used for financial reporting and tenant billing.

RoomBaseDetails - meter_readings Relationship:

The meter_readings table also utilizes a room_id foreign key to establish a one-to- many relationship with the RoomBaseDetails table. This relationship is used to maintain a log of all utility meter readings such as water and electricity for each room, which is essential for accurate billing and consumption tracking.

RoomBaseDetails – bills Relationship:

The bills table has a room_id foreign key that points to the RoomBaseDetails table, creating one-to-many relationship. Each room can have multiple associated bills that detail the costs incurred by the tenant for utilities, additional rates, and other relevant charges, ensuring that each cost is accounted for and can be queried per room.

RoomBaseDetails – Requests Relationship:

The Request table's room_id foreign key references the RoomBaseDetails table's room_id in a one-to-many relationship. This allows for multiple requests or issues to be recorded and tracked for each room, such as maintenance or service requests, which is vital for property management and maintenance scheduling.

RoomStatusDetails Table

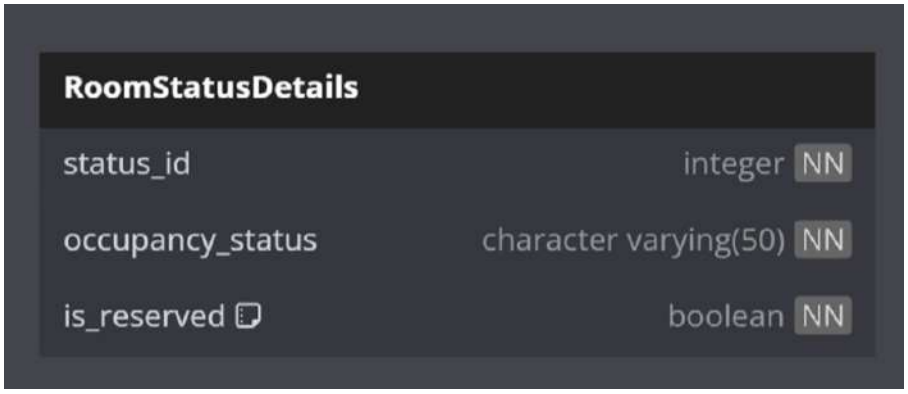


Figure: 7.9 RoomStatusDetails Table

RoomStatusDetails Table Field:

status_id: Unique identifier for each room status.
occupancy_status: Description of the room's occupancy status.
is_reserved: Indicates if the room is reserved or not.

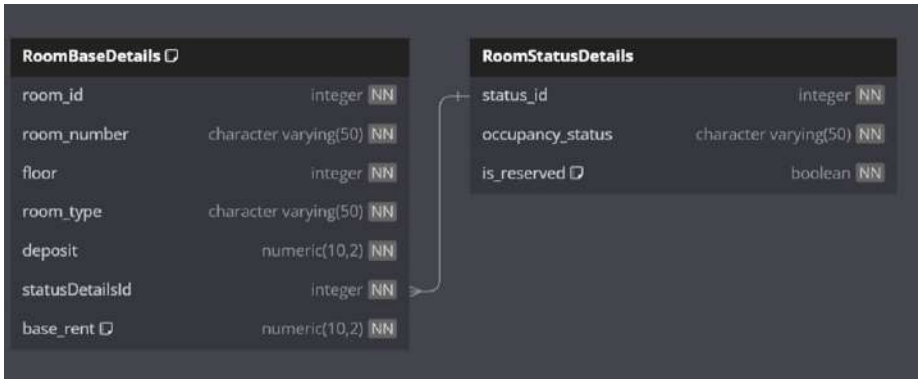


Figure: 7.10 RoomStatusDetails Table's Relationship

RoomStatusDetails – RoomBaseDetails Relationship:

Through the statusDetailsId field in the RoomBaseDetails, which corresponds to the status_id in the RoomStatusDetails table, a one-to-one relationship is created. Each room is linked to a single status detail record, which would include information on whether the room is occupied, vacant, or reserved.

Tenancy_records Table

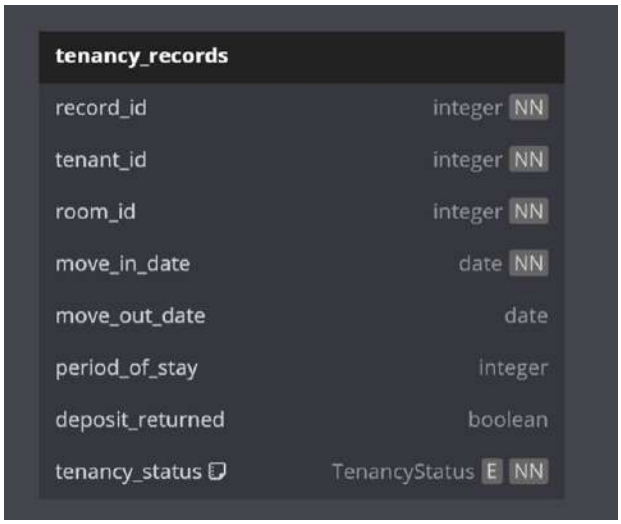


Figure: 7.11 Tenancy_records Table

Tenancy_records Table Fields:
record_id: Unique ID for the tenancy record.
tenant_id: Unique ID of the tenant.
room_id: Unique ID of the rented room.
move_in_date: Date tenant moved in.
move_out_date: Date tenant moved out.
period_of_stay: Length of stay in numerical terms.
deposit_returned: Whether the deposit was returned (true/false).
tenancy_status: Status of tenancy, which can be either 'CHECK_IN' (tenant has moved in) or 'CHECK_OUT' (tenant has moved out), as defined by the TenancyStatus enum.

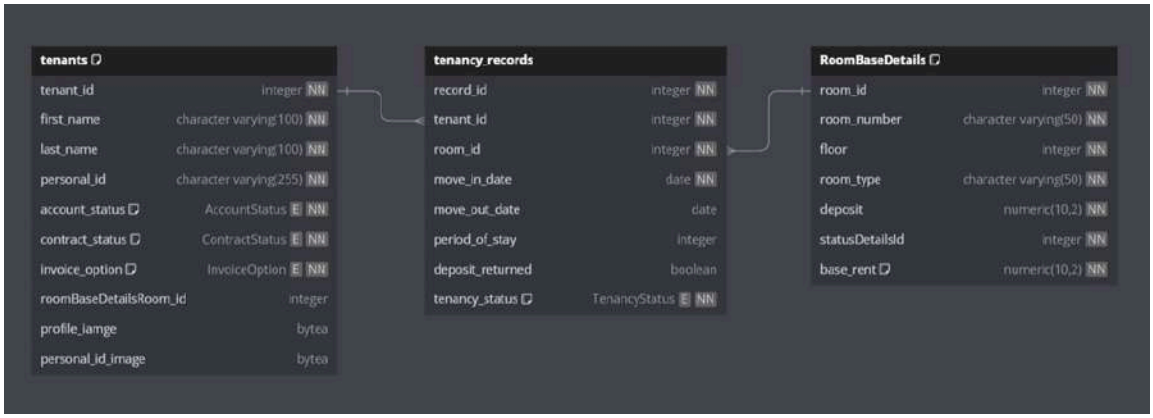


Figure: 7.12 Tenancy_records Table's Relationships

Tenancy_records – Tenants Relationship:

This is a many-to-one relationship, meaning multiple tenancy records can be associated with a single tenant.

Tenancy_records – RoomBaseDetails Relationship:

This relationship is a one-to-one relationship meaning that it is always associated with a single tenancy record at a time.

Rates Table

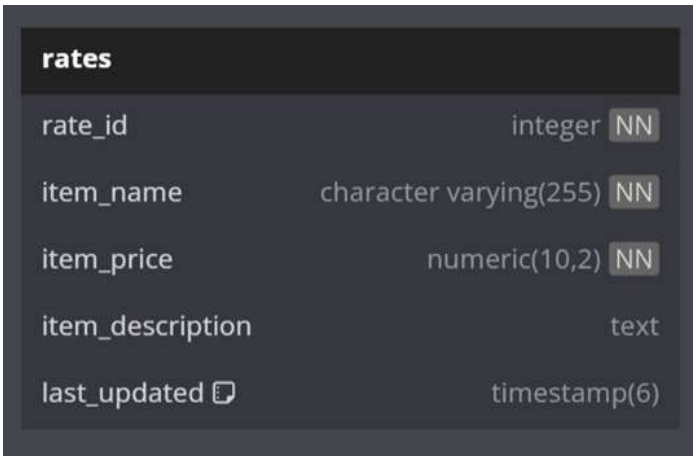


Figure 7.13 Rates Table

Rates Table Fields:
rate_id: Unique identifier for each rate.
item_name: Name or description of the rate item.
item_price: Price of the rate item.
item_description: Additional description or information about the rate item.
last_updated: Timestamp indicating when the rate was last updated with a default value of the current timestamp.

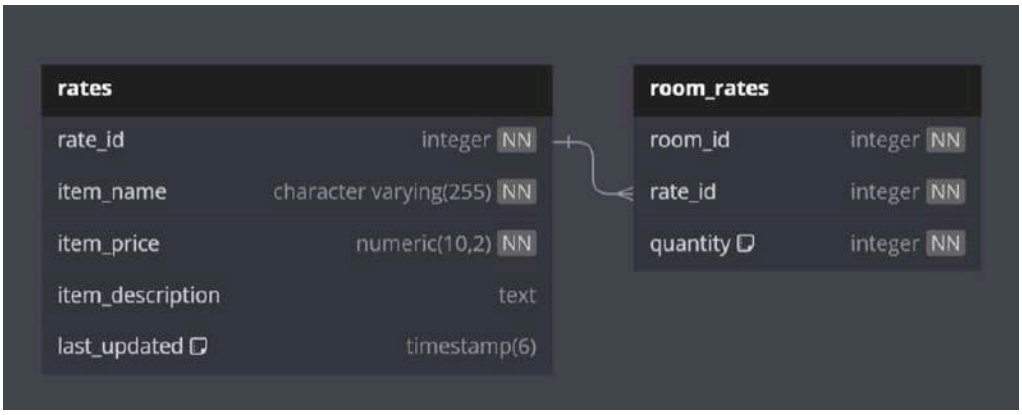


Figure: 7.14 Rates Table's Relationship

Rates – Room_Rates Relationship:

This is a many-to-one relationship, where multiple entries in room_rates can reference a single rate_id from the rates table. This setup allows for multiple rooms to be associated with the same rate.

Room_rates Table

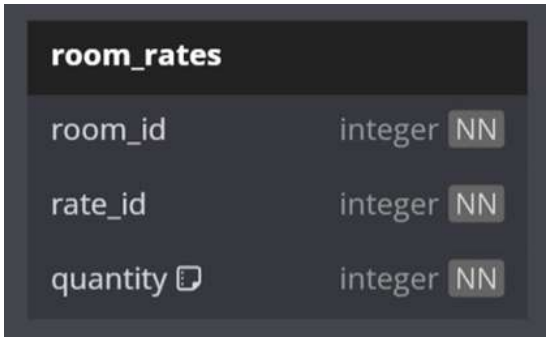


Figure: 7.15 Room_rates Table

Room_rates Table Fields:
room_id: Identifier for a room.
rate_id: Identifier for a rate item associated with the room.
quantity: Quantity of the rate item for the room.

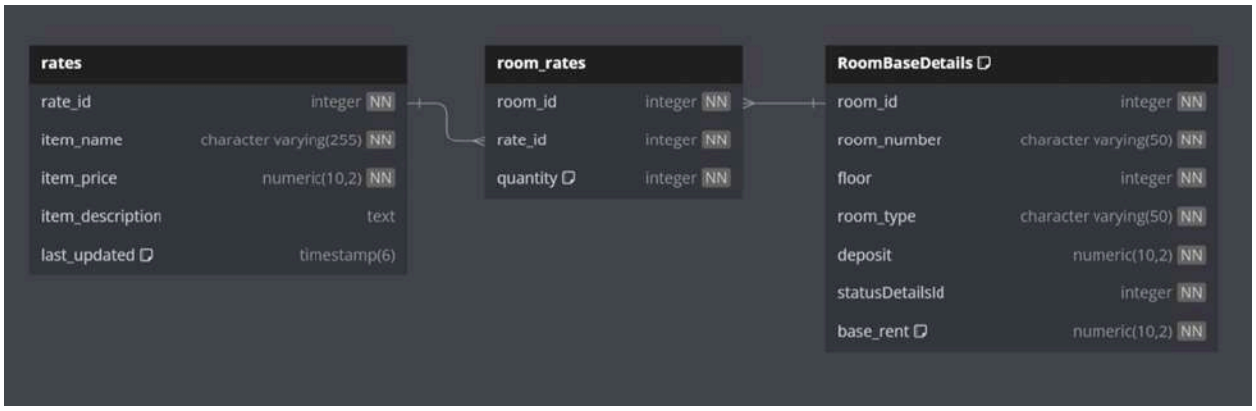


Figure: 7.16 Room_rates Table's Relationship

Room_rates Table's Relationship:

This is a many-to-one relationship, where multiple entries in room_rates can reference a single rate_id from the rates table. This setup allows for multiple rooms to be associated with the same rate.

Room_rates - RoomBaseDetails Relationship:

There is a one-to-many relationship established by the room_id foreign key in the room_rates table. This indicates that each room can have various rates applied over time, possibly to account for peak pricing, discounts, or other rate adjustments.

Meter_readings Table



Figure: 7.17 Meter_readings Table

Meter_readings Table Fields:
reading_id: Unique identifier for each meter reading.
room_id: Identifier for the room associated with the reading.
water_reading: Recorded water meter reading.
electricity_reading: Recorded electricity meter reading.
reading_date: Date when the readings were recorded.

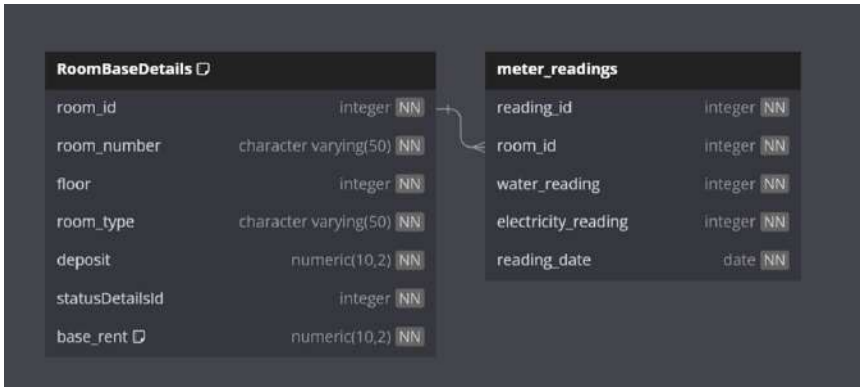


Figure: 7.18 Meter_readings Table's Relationship

Meter_readings – RoomBaseDetails Relationship:

The meter_readings table utilizes a room_id foreign key to establish a one-to-many relationship with the RoomBaseDetails table. This relationship is used to maintain a log of all utility meter readings such as water and electricity for each room, which is essential for accurate billing and consumption tracking.

Requests Table

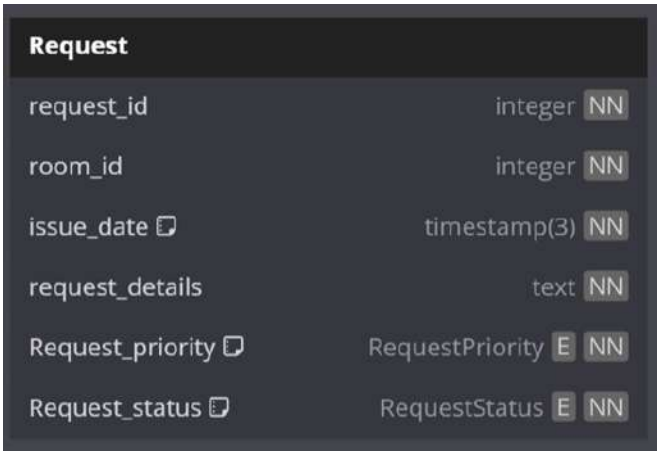


Figure: 7.19 Requests Table

Requests Table Fields:
request_id: Unique identifier for each request.
room_id: Identifier for the room associated with the request.
issue_date: Timestamp indicating when the request was issued with a default value of the current timestamp.
request_details: Details or description of the request.
Request_priority: Priority level of the request, which can be one of the following: 'URGENT,' 'HIGH,' 'MODERATE,' 'NORMAL,' 'LOW'.
Request_status: Status of the request, which can be either 'RESOLVED' or 'UNRESOLVED'.

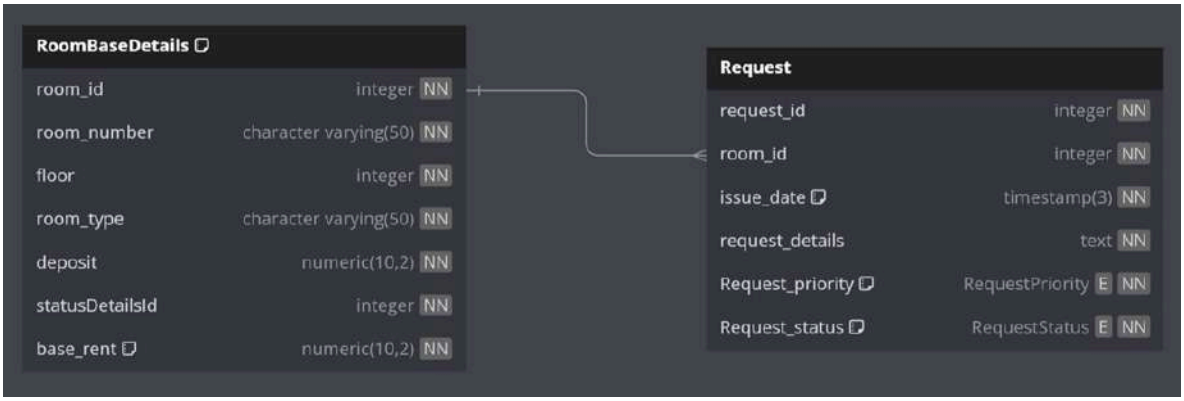


Figure: 7.20 Requests Table's Relationship

Requests – RoomBaseDetails Relationship:

The Request table's room_id foreign key references the RoomBaseDetails table's room_id in a one-to-many relationship. This allows for multiple requests or issues to be recorded and tracked for each room, such as maintenance or service requests, which is vital for property management and maintenance scheduling.

Bills Table

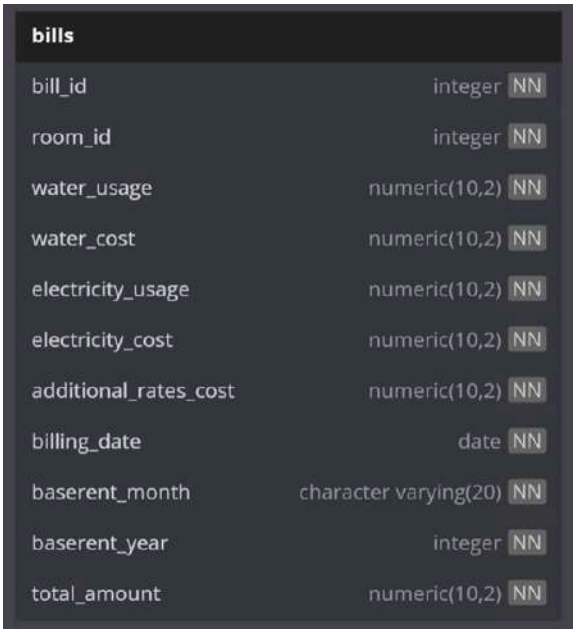


Figure: 7.21 Bills Table

Bills Table Fields:
bill_id: Unique identifier for each bill.
room_id: Identifier for the room associated with the bill.
water_usage: Recorded water usage.
water_cost: Cost associated with water usage.
electricity_usage: Recorded electricity usage.
electricity_cost: Cost associated with electricity usage.
additional_rates_cost: Cost of additional rates.
billing_date: Date when the bill was generated.
baserent_month: Month for the base rent.
baserent_year: Year for the base rent.
total_amount: Total amount of the bill.

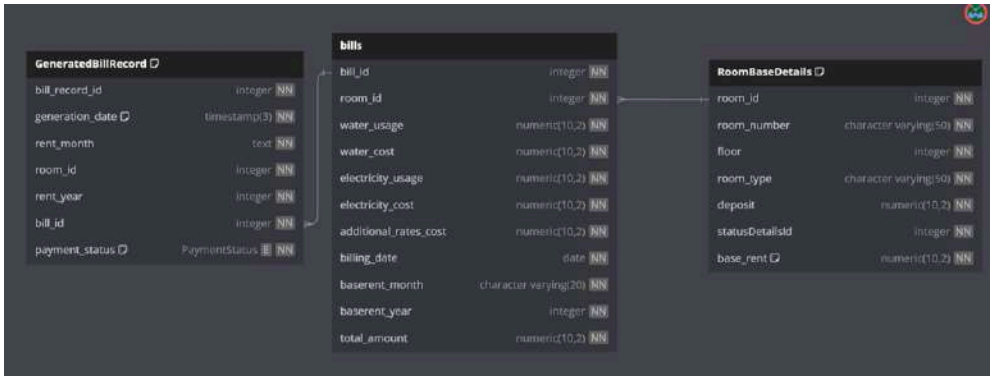


Figure: 7.22 Bills Table's Relationships

Bills – GeneratedbillRecords Relationship:

There is a one-to-one relationship between the GeneratedbillRecords, and bills tables established by the bill_id foreign key in the GeneratedBillRecord table, indicating that each generated bill record corresponds to exactly one bill entry.

Bills – RoomBaseDetails Relationship:

There is a one-to-many relationship between the RoomBaseDetails and bills tables established by the room_id foreign key in the bills table. This indicates that each room can have multiple bills associated with it over time, reflecting various billing periods and charges.

GeneratedbillRecords Table

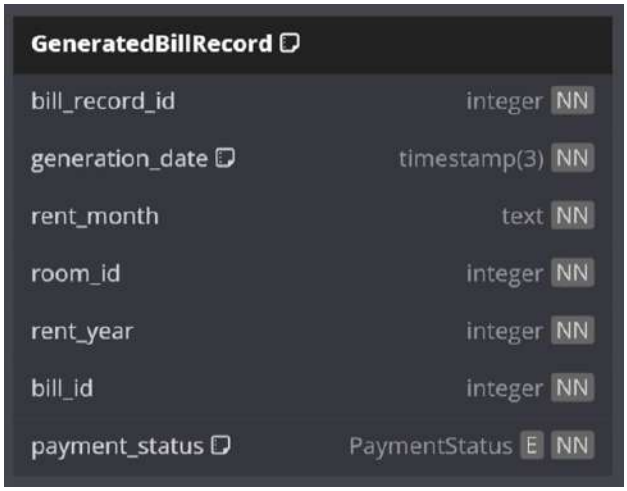


Figure: 7.23 GeneratedbillRecords Table

GeneratedbillRecords Table fields:
bill_record_id: Unique identifier for each generated bill record.
generation_date: Timestamp indicating when the bill record was generated with a default value of the current timestamp.
room_id: Identifier for the room associated with the bill record.
rent_month: Month for which the rent is calculated.
rent_year: Year for which the rent is calculated.
bill_id: Identifier for the associated bill.
payment_status: Payment status of the bill record, which can be one of the following: 'PAID,' 'PENDING,' 'OVERDUE,' 'PARTIAL'.

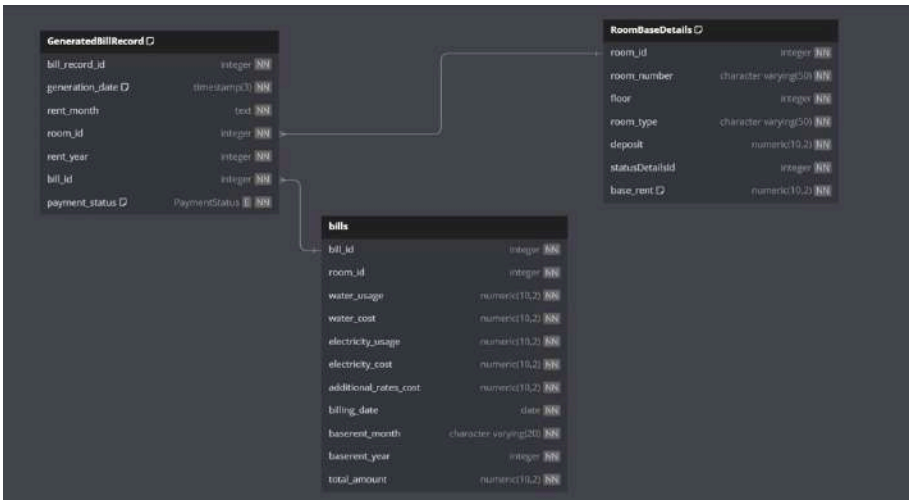


Figure: 7.24 GeneratedbillRecords Table's Relationships

GeneratedbillRecords – Bills Relationship:

There is a one-to-one relationship established by the bill_id foreign key in the GeneratedBillRecord table, which means each bill record corresponds to exactly one bill from the bills table.

GeneratedbillRecords – RoomBaseDetails Relationship:

There is a one-to-many relationship established by the room_id foreign key in the bills table with respect to the RoomBaseDetails table. This indicates that each room can be associated with multiple bills over time, reflecting different billing periods and charges.

Manager Table

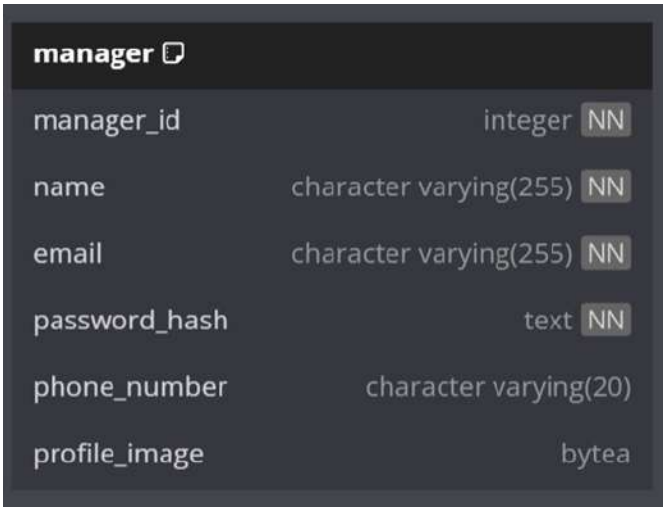


Figure: 7.26 Manager Table

Manager Table Fields:
manager_id: Unique identifier for each manager.
name: Manager's name.
email: Manager's email address.
password_hash: Hashed password for manager's account. phone_number: Manager's phone number.
profile_image: Binary data for manager's profile image.



Figure: 7.27 Manager Table's Relationship

Manager – Service_Contacts Relationship:

There is a one-to-many relationship between the manager and service_contacts tables, established by the manager_id foreign key in the service_contacts table. This indicates that each manager can have multiple service contacts associated with them, but each service contact is linked to only one manager.

Service_Contacts Table

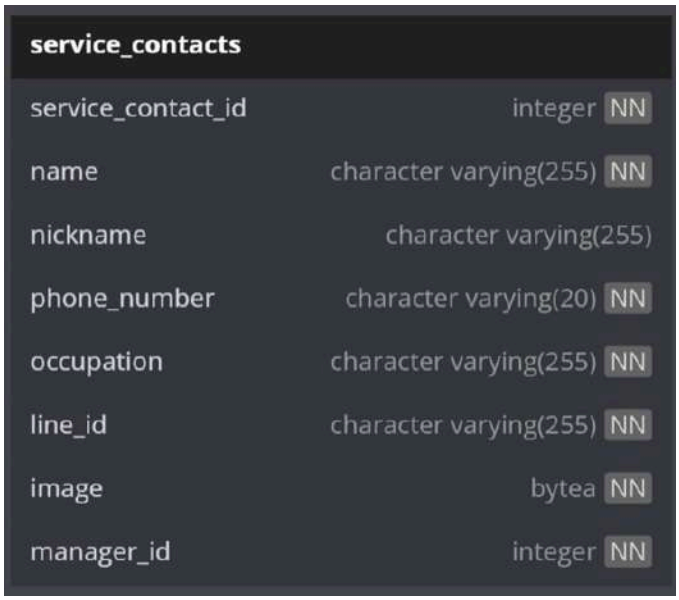


Figure: 7.28 Service_Contacts Table

Service_Contacts Table Fields:
service_contact_id: Unique identifier for each service contact.
name: Name of the service contact.
nickname: Nickname for the service contact.
phone_number: Phone number of the service contact.
occupation: Occupation of the service contact.
line_id: Line ID of the service contact.
image: Binary data representing the image of the service contact. manager_id: Identifier for the associated manager.



Figure: 7.29 Service_Contacts Table's Relationship

Service_Contacts – Manager Relationship:

There is a one-to-many relationship between the manager and service_contacts tables, established by the manager_id foreign key in the service_contacts table. This indicates that manager can have multiple service contacts associated with them, but each service contact is linked to only one manager.

Chapter Eight: Final Version Interface Design

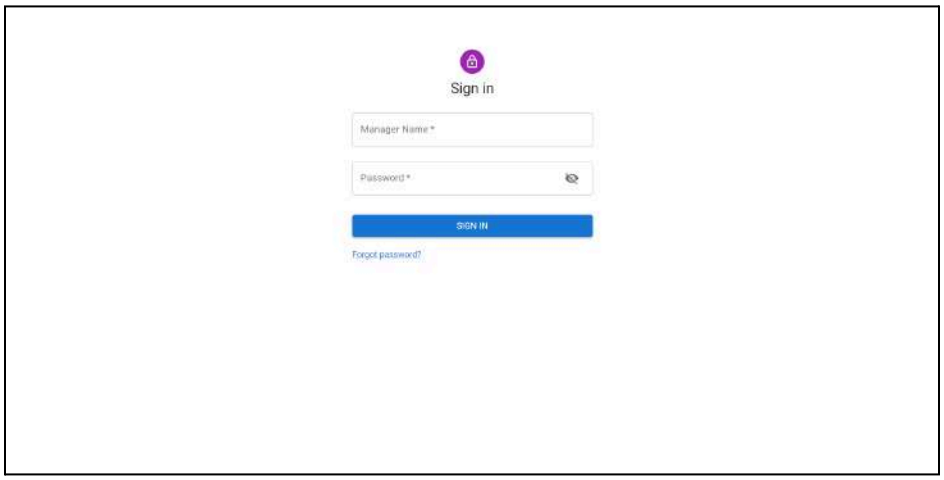


Figure 8.1: Sign-In Interface

This figure shows the initial sign-in screen for the PS Park Apartment Management System, where managers can enter their credentials to access the system. Key features include fields for manager name and password, and links for account recovery options.

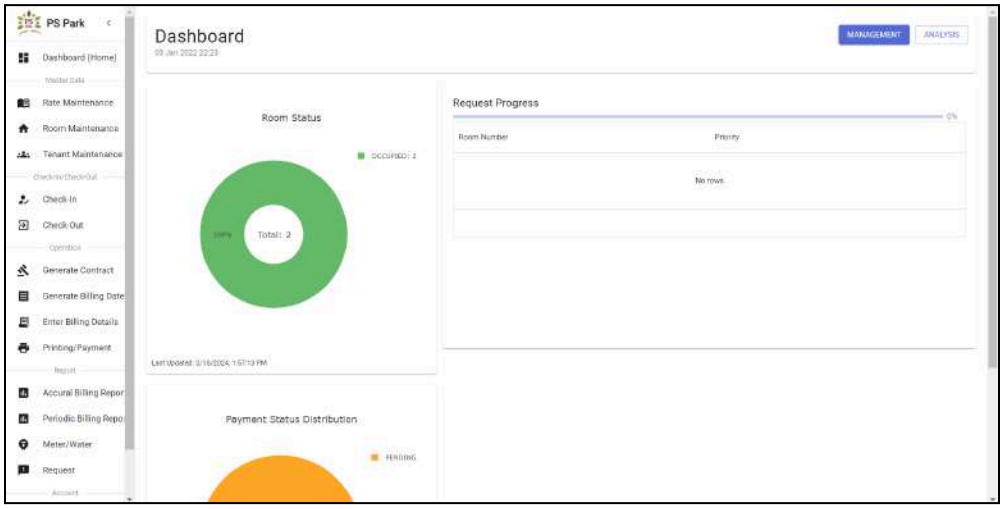


Figure 8.2: Dashboard (Management)

This figure shows all the information and activities that is going on in the buildings. By presenting them with graphs. Things such as, room status, recent payment, request table, and contact tables.

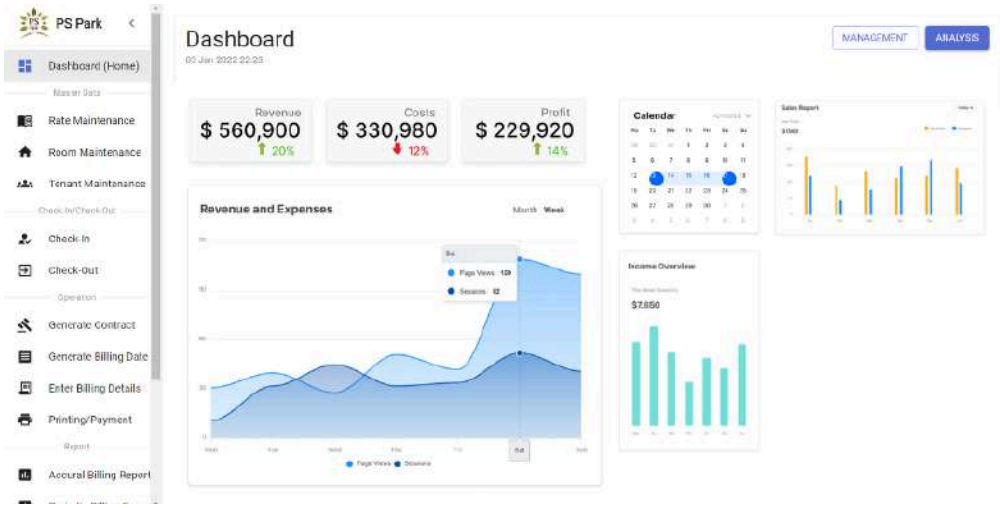


Figure 8.3: Dashboard (Analysis)

This figure shows all the financial activities of the building. By presenting them with graphs. Things such as, revenue and expenses, costs, profit, sales report, calender and income overview.

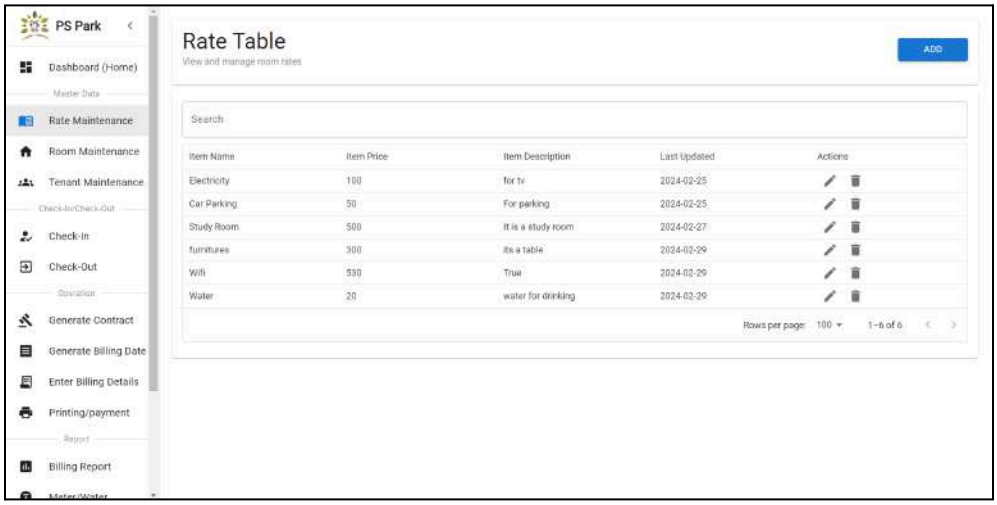


Figure 8.4: Rate Table Interface

The rate table interface is displayed here, providing a comprehensive view and management options for different rates associated with the apartment such as electricity, car parking, and furniture, with details on pricing and descriptions.

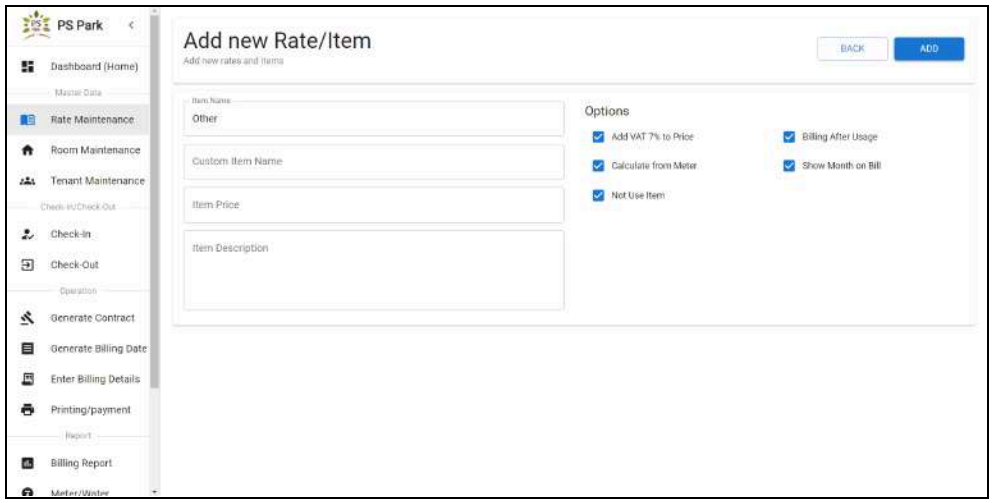


Figure 8.5: Add New Rate/Item Interface

This figure shows the interface for adding new rates or items within the PS Park system. It includes input fields for item name, price, and description, alongside options for VAT addition, billing after usage, and other financial calculations.

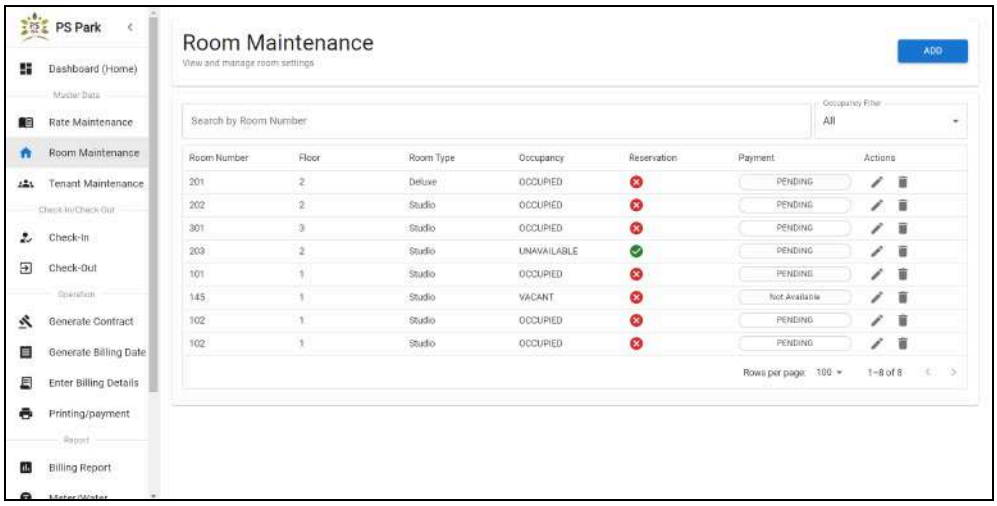


Figure 8.6: Room Table Interface

The room table interface is displayed here, providing a comprehensive the room number, floor, room type, its occupancy status, reservation status, and payment status.

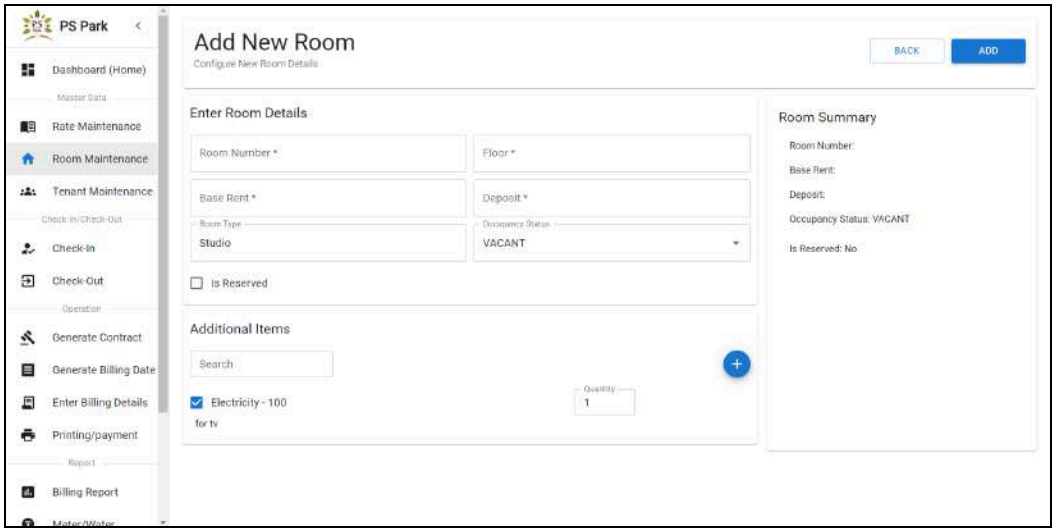


Figure 8.7: Add New Room Interface

This figure illustrates the interface used for adding new rooms to the system. It provides fields for room number, base rent, and deposit, as well as the ability to assign additional items and services to each room.

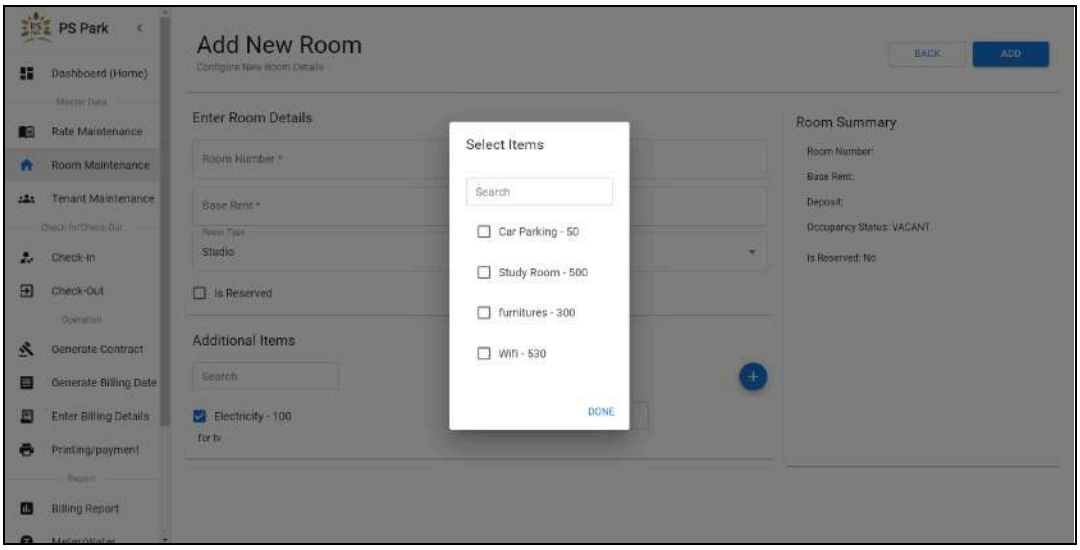


Figure 8.8: Room Item Selection Interface

The room item selection interface is shown here, where items such as car parking, study room access, Wi-Fi services and many more can be added to a room, each with associated costs.

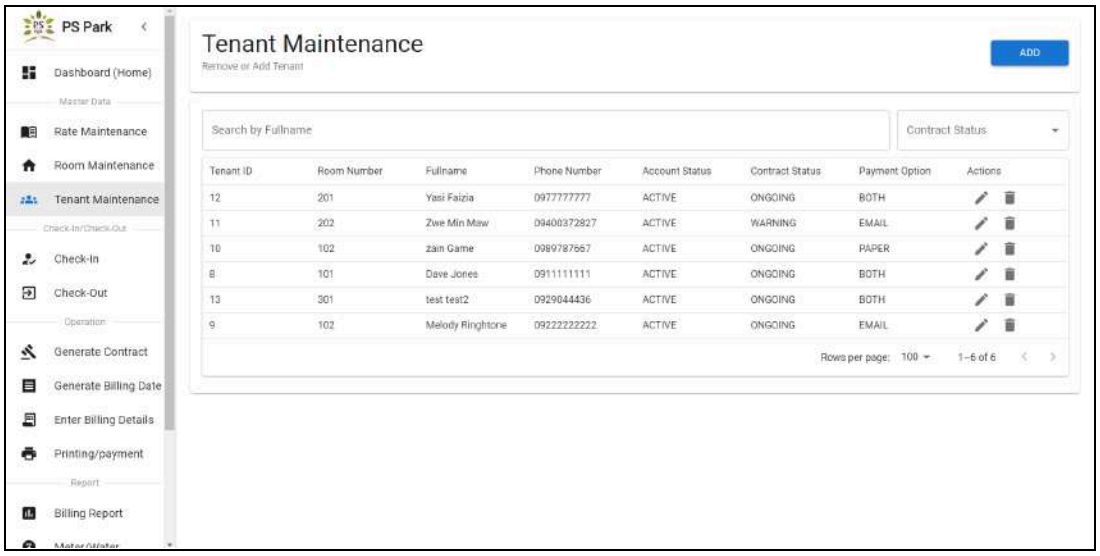


Figure 8.9: Tenant Table Interface

The tenant table interface depicted here is where tenants can seen with their ID, Number, Name, Phone Number, Account Status, Contract Status, and Payment Option to the system.

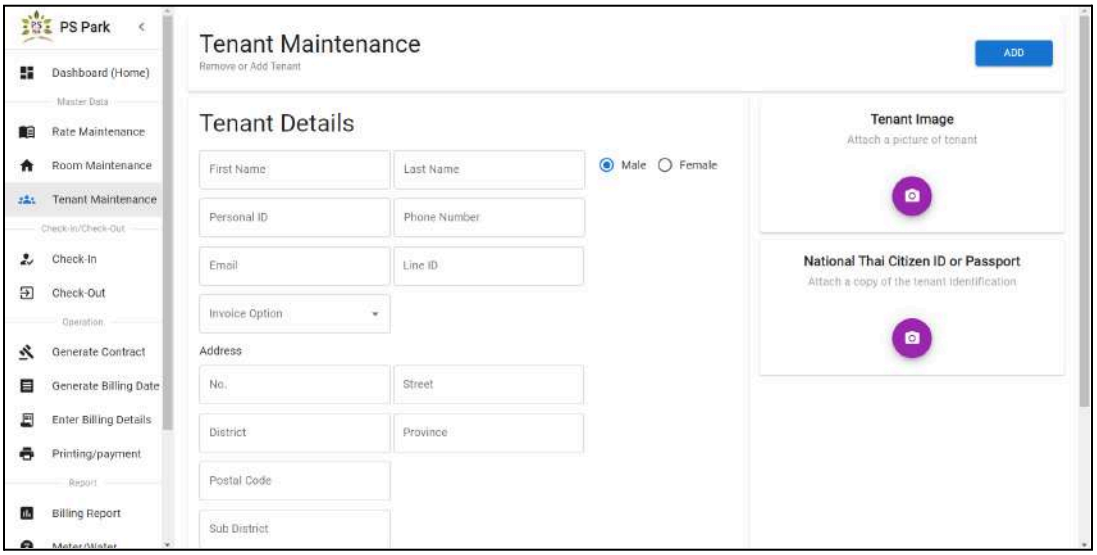


Figure 8.10: Tenant Maintenance Interface

The tenant maintenance interface depicted here is where new tenants can be added to the system, with fields for personal information, contact details, and document attachments.

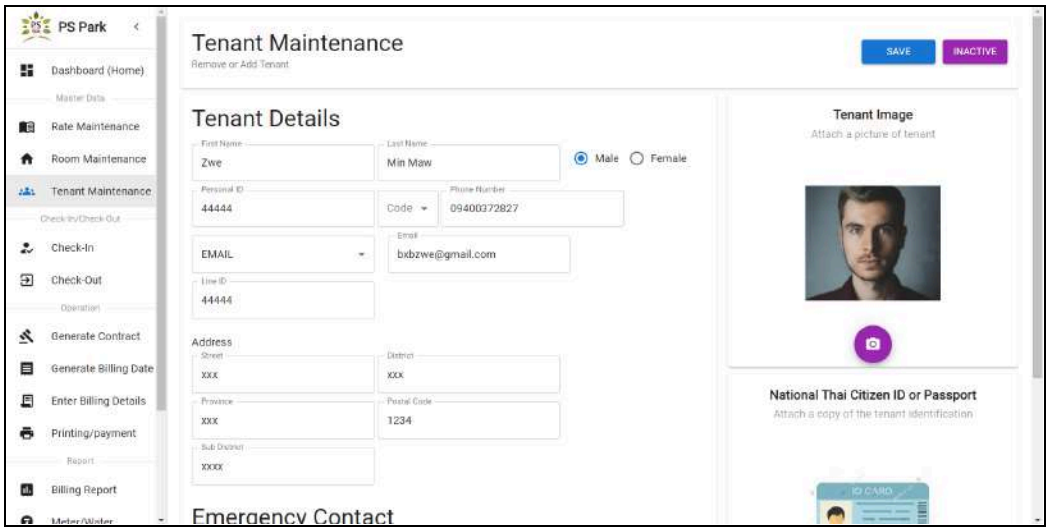


Figure 8.11: Tenant Maintenance Interface Edit

The tenant maintenance interface depicted here is where the old tenants can be edited to the system, with fields for personal information, contact details, and document attachments.

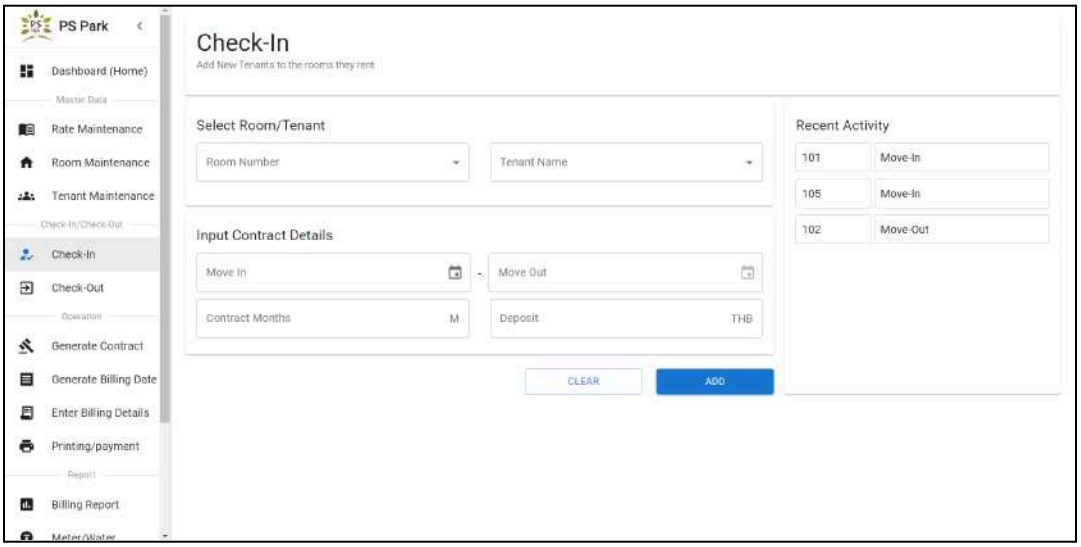


Figure 8.12: Check-In

This page interface of the PS Park management system, showing fields to input room number, tenant name, move-in date, contract duration, and deposit amount. On the right, the recent activity log displays recent move-ins and move-outs for tracking changes.

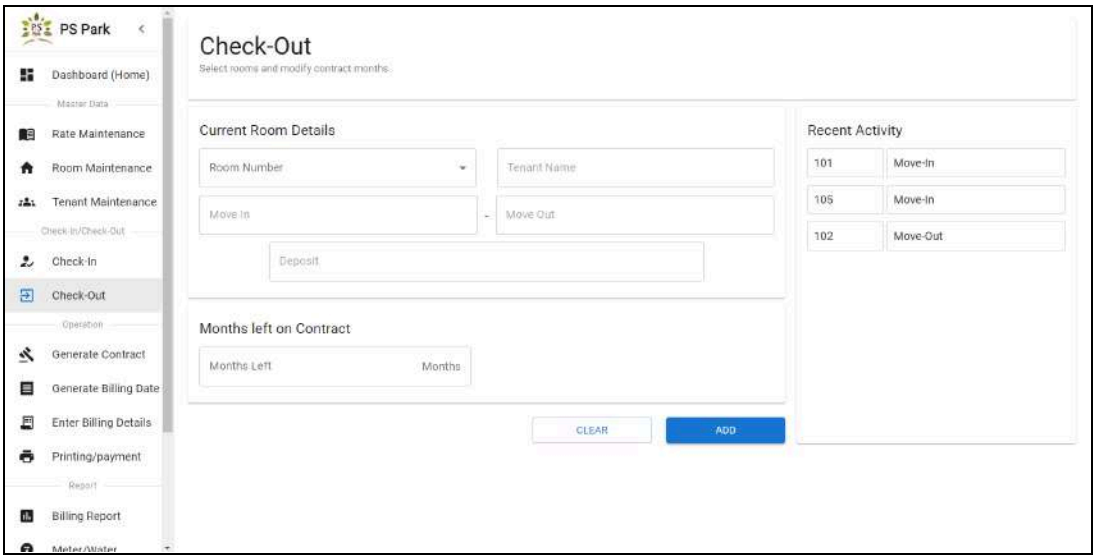


Figure 8.13: Check-Out

This page interface displaying fields for current room details, including room number, tenant name, move-in date, deposit amount, and the number of months left on the contract. The recent activity log is shown on the right side for quick reference.

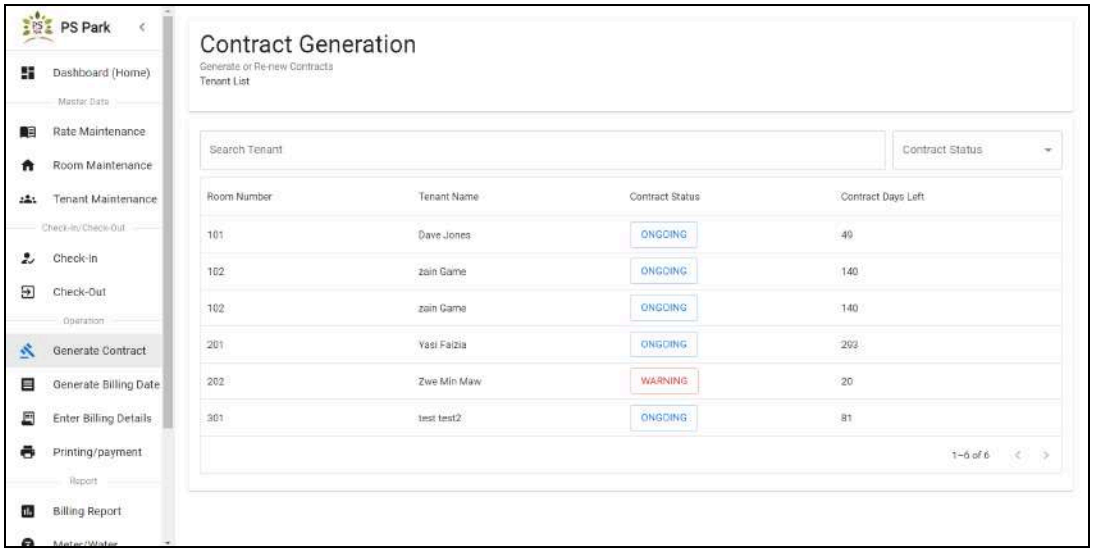


Figure 8.14: Contract Generation

This screen where users can generate or renew contracts. The interface lists tenant names with room numbers, contract status, and remaining days. Options for updating the period of stay and generating the contract in English or Thai are provided.

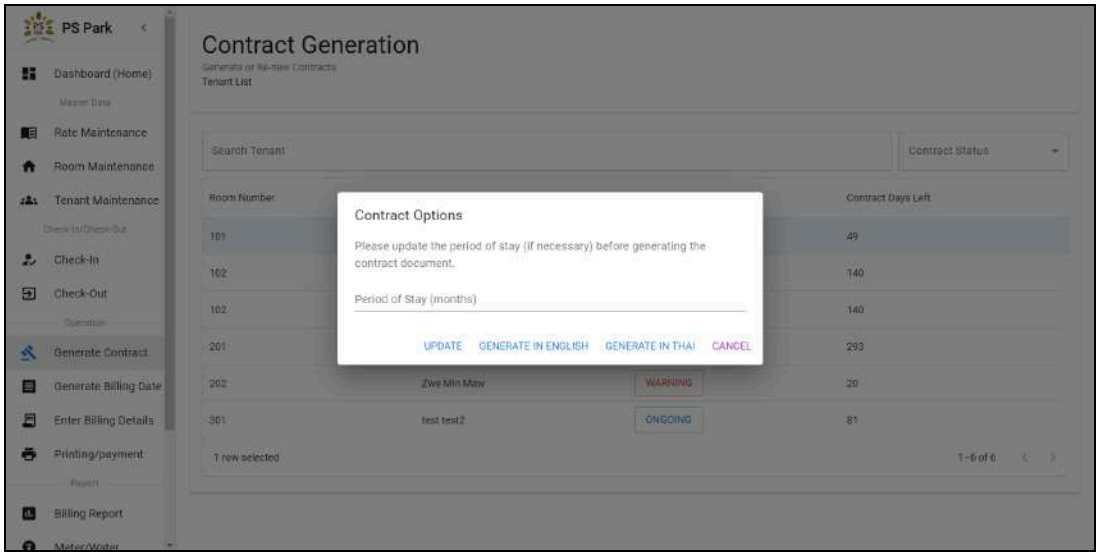


Figure 8.15: Contract Generation (Contract Options)

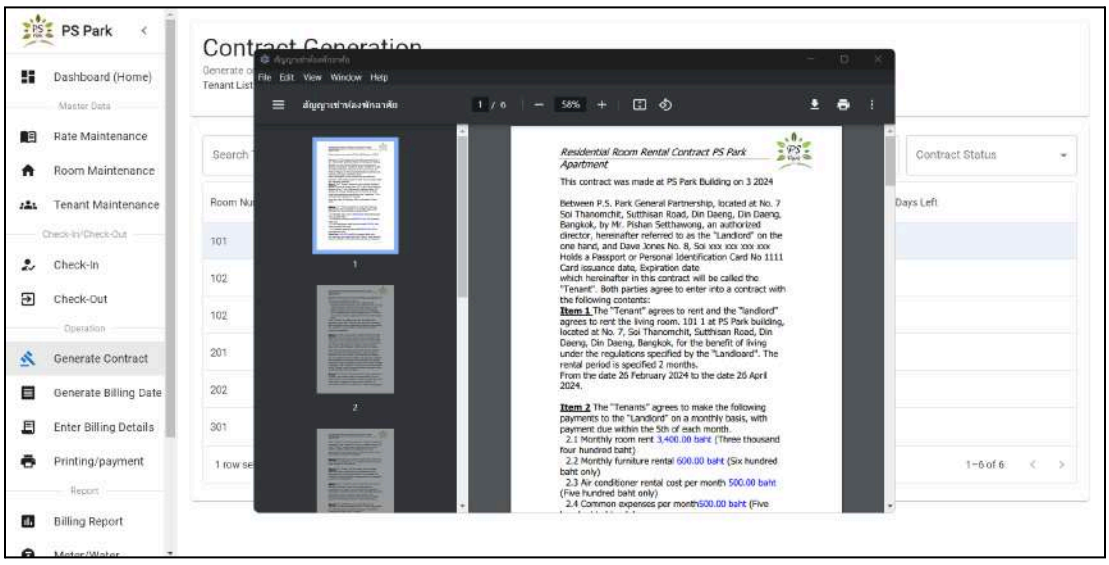


Figure 8.16: The PDF Generator

Generating contract with specific tenants being chosen by the manager.

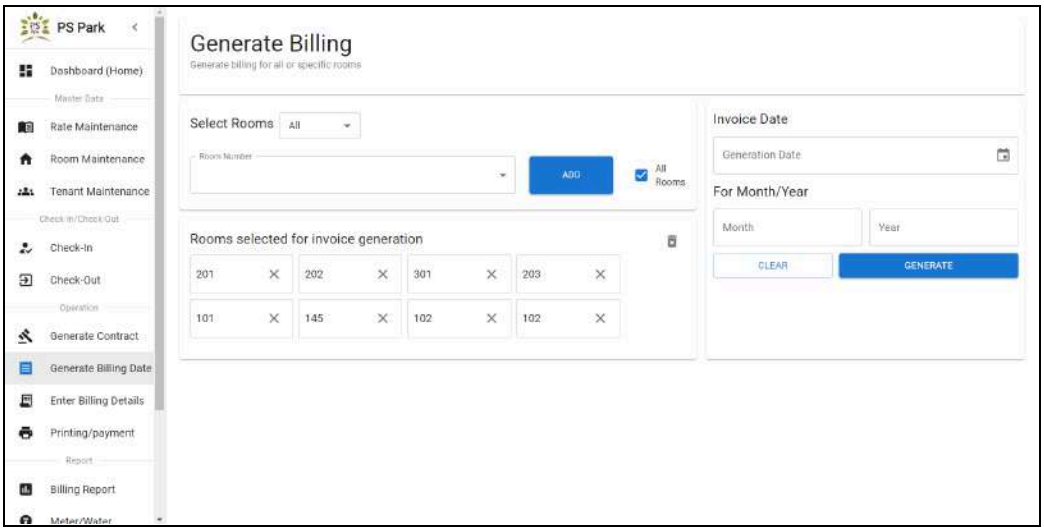


Figure 8.17: Generate Billing

The interface, allowing the selection of individual rooms or all rooms to generate billing. Room numbers are listed once selected, with an option to input the invoice date and specify the billing month and year.

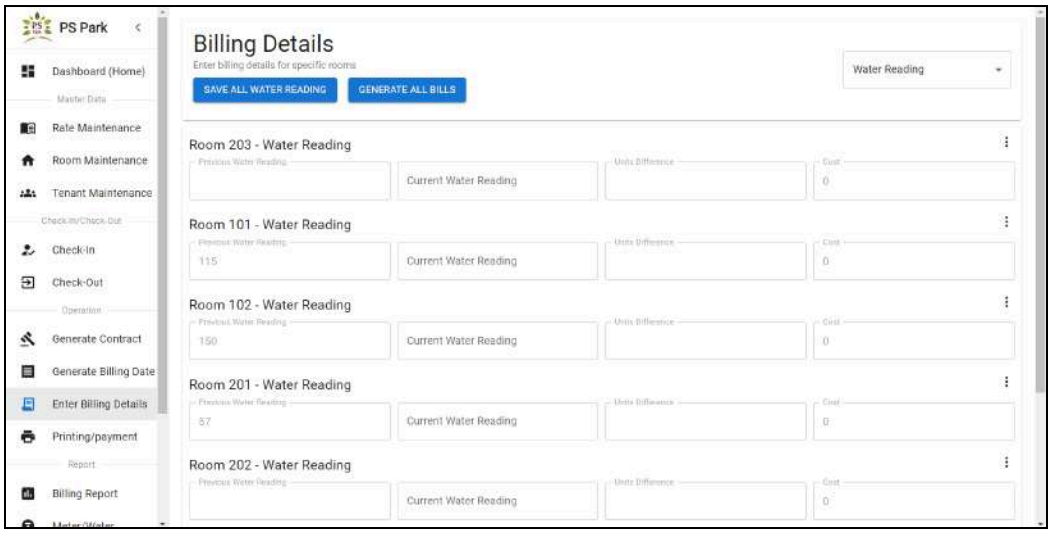


Figure 8.18: Billing Details

This section where the specific rooms' water readings can be entered, showing previous readings, the current readings, units used, and the cost calculated based on consumption for both water and electricity readings.

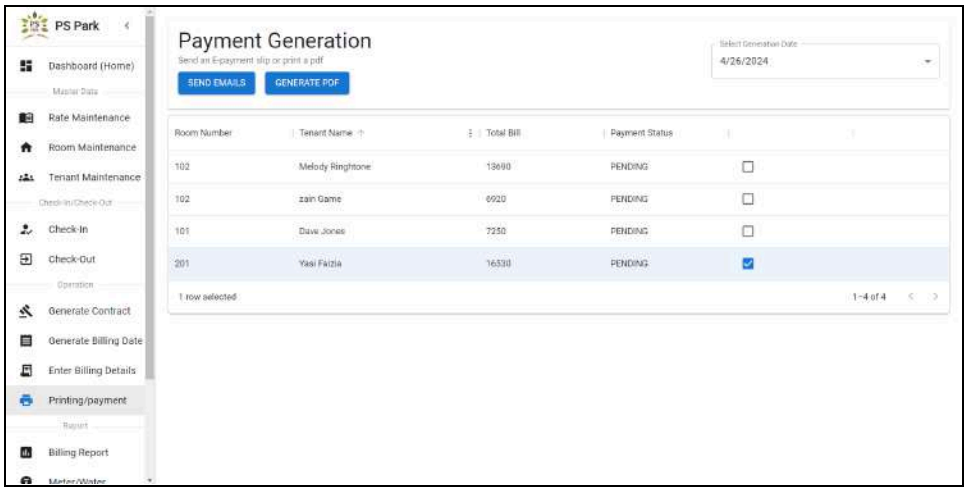
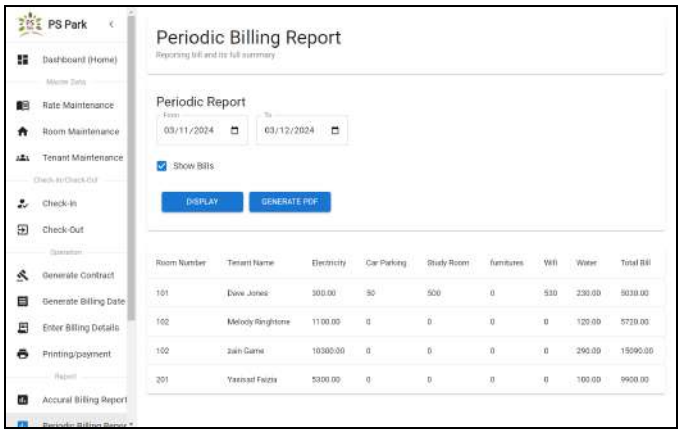


Figure 8.19: Payment Generation

This page, designed to send E-payment slips or generate a PDF of the payment details. Room numbers, tenant names, total bill, and payment status are shown to the manager, with options to send emails or generate PDFs.



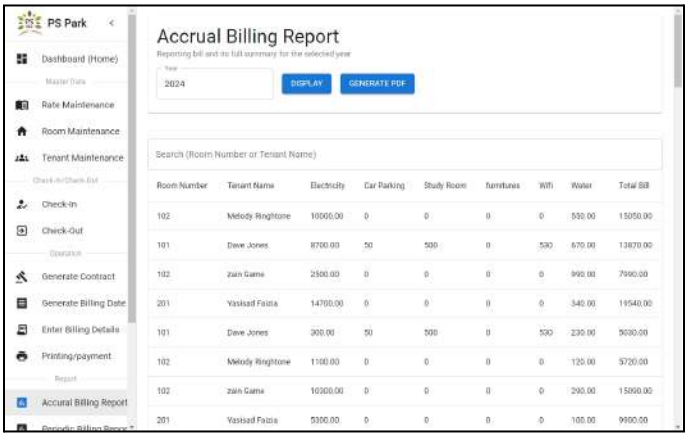


Figure 8.20: Billing Report

This interface, providing options to generate periodic or accrual reports, with checkboxes to display bills, tax invoices, and unpaid bills within specified date ranges.

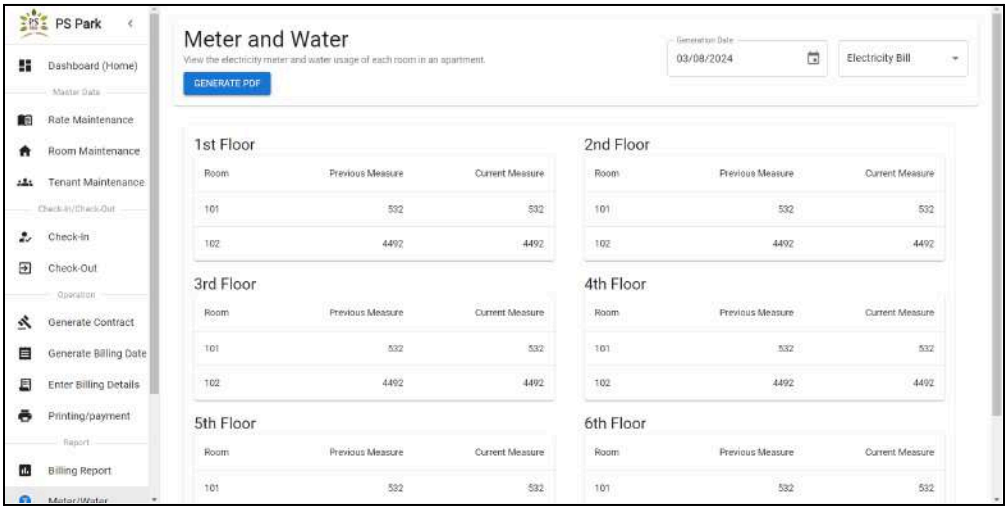


Figure 8.21: Meter and Water

The summary screen displaying electricity and water meter readings for each room, listed by floor. It includes previous and current readings, facilitating the generation of a PDF report for records or billing purposes.

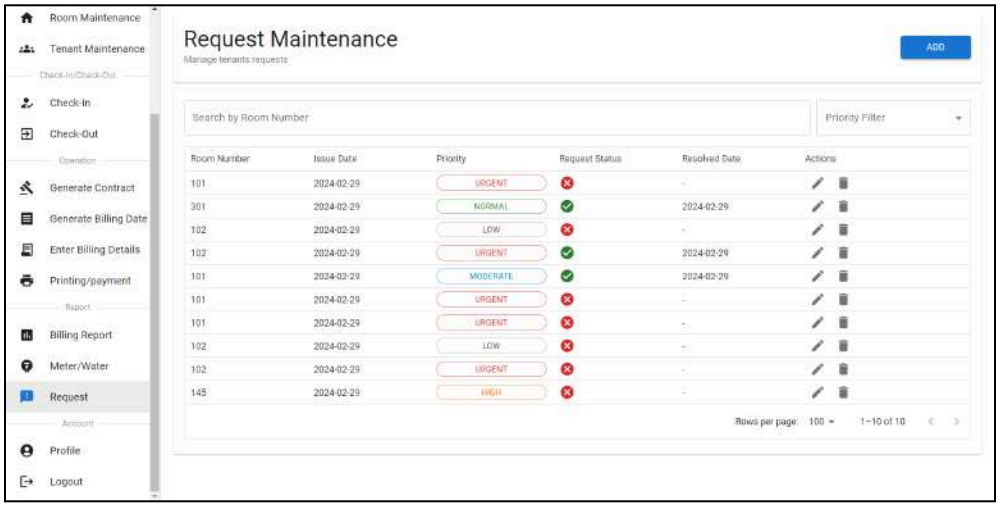


Figure 8.22: Request Maintenance

This screen displays a table of maintenance requests with columns for room number, issue date,

priority, request status, resolved date, and actions. The priority levels are color-coded for urgency, with red indicating urgent requests.

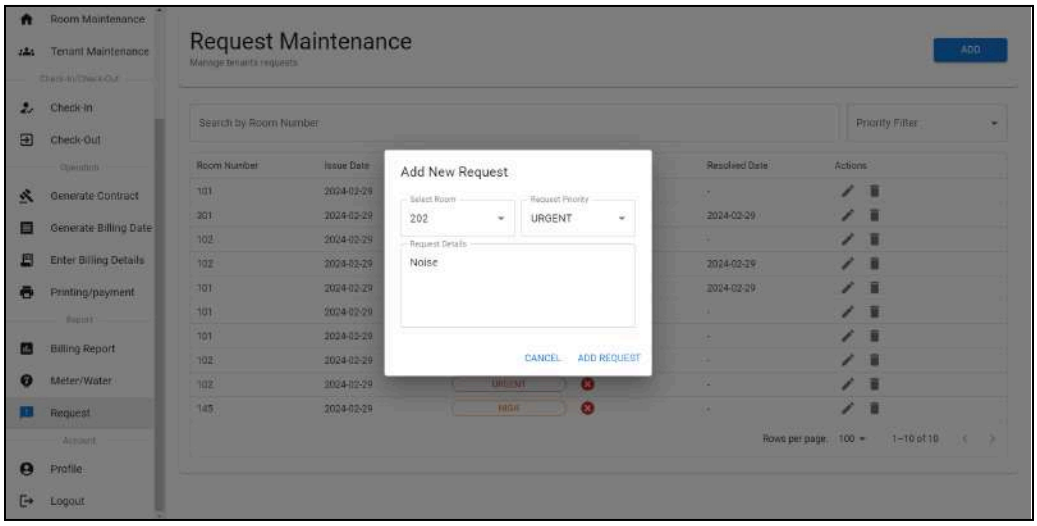


Figure 8.23: interface for adding a new maintenance request. It provides a form for selecting the room number, setting the priority of the request, and describing the request details.

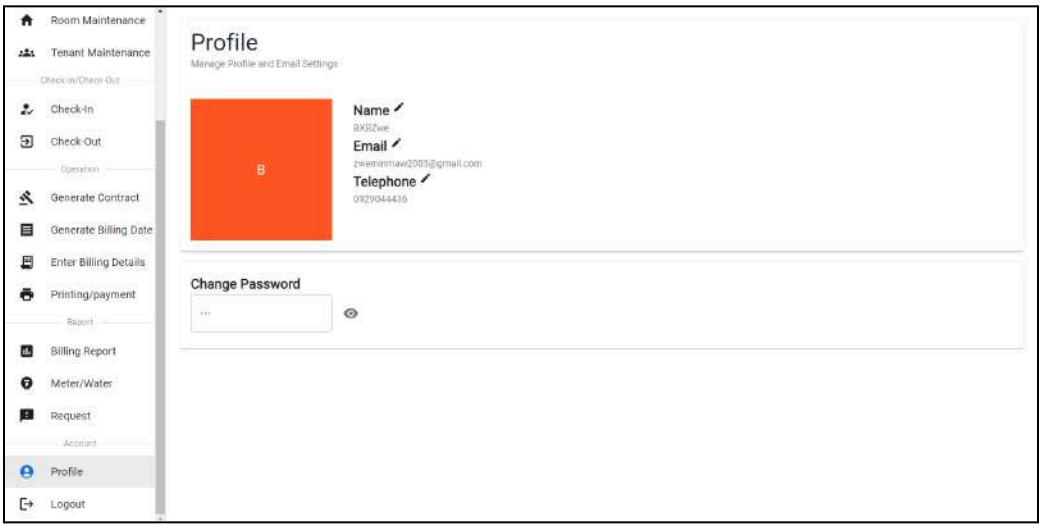


Figure 8.24: Profile

This section where users can view their profile information, including name, email, and telephone number.

Chapter Nine: Conclusion

In conclusion, the development of the PS Park application was driven by a distinct set of objectives: streamlining property management processes, enhancing security measures, and fostering financial transparency. The integration of advanced functionalities alongside a user-friendly interface underscores the application's capacity to revolutionize property management within the real estate sector. This project, enriched by the collaborative efforts with stakeholders, including our project advisor, Asst. Prof. Dr. Rachsuda Setthawong, Asst. Prof. Dr. Pisal Setthawong, the Apartment Manager, Mr. Pishan Setthawongand and the committee members, Mr. Chayapol Moemeng and Asst. Prof. Dr. Anilkumar Kothalil Gopalakrishnan, has been a testament to the transformative potential of software solutions in improving operational efficiency, security, and transparency in apartment complexes and residential communities. Their guidance and expertise were pivotal in navigating the challenges encountered, ensuring the project's success. The PS Park application stands as a significant advancement in property management, promising a more efficient, secure, and transparent operation.

Reference:

1. PostgreSQL “Database” Documentation. https://www.postgresql.org/docs/
2. Nextron “Framework” Documenation. https://github.com/saltyshiomix/nextron
3. Libreoffice “Contract Generation” https://www.npmjs.com/package/libreoffice-convert https://help.libreoffice.org/latest/lo/text/shared/autopi/01130000.html
4. Node.js “Runtime Environment” Documentation.
5. Danfo.js “Data Structure” Documentation. https://danfo.jsdata.org/
6. Material-UI “User Interface” Documentation. https://mui.com/material-ui/getting-started/